CODE//CONNECT'26 · VIBE CODING HACKATHON · PROBLEM 01: DISASTER RESPONSE

PulseTriage

Finding the people who can't call for help

Team brief: what the problem is, what we are building, how we warn people and hear back from them, how every part works, and exactly how we build it, step by step.

Floods | Heatwaves | Earthquakes | Storms and cyclones | Wildfires | Landslides

---

How to read this

- **Everyone, 20 minutes:** Part 1 (the problem) and Part 2 (our solution).
- **Builders:** Part 3 (how it works) and Part 4 (how we build it). Part 4 has your hour-by-hour task list and ready-to-paste prompts for your AI coding tool.
- **Before the demo:** Part 5 (demo and pitch) and Part 7 (risks and the day itself).
- **Stuck on a word?** The glossary at the end explains every technical term in plain language.

Version 4 · 17 October 2026 · Covers every disaster type, warnings and reporting, team and resource tracking, and direct upload with phone-to-phone relay as the fallback. All numbers marked as demo defaults are ours to tune; nothing here is medical advice.

Contents

Part 1. The problem

- 1.1 The hackathon in one minute
- 1.2 Problem 01 in plain words
- 1.3 The hidden trap: the quietest are in the most danger
- 1.4 Two gaps: warnings out, calls for help in
- 1.5 What happens today

Part 2. Our solution: PulseTriage

- 2.1 PulseTriage in one paragraph
- 2.2 The loop: warn, hear back, rank, act
- 2.3 One system for every disaster
- 2.4 Where the idea came from
- 2.5 The two apps
- 2.6 The seven ideas that make it different
- 2.7 How we answer every part of the problem statement
- 2.8 What we are deliberately not building

Part 3. How it works

- 3.1 The big picture
- 3.2 Warning people: how alerts reach everyone
- 3.3 Hearing back: every way a report comes in
- 3.4 Two journeys: a flood and a heatwave
- 3.5 The Field app
- 3.6 The Command dashboard
- 3.7 Tracking teams and resources
- 3.8 How phones pass messages to each other (the mesh)
- 3.9 The brain: hazard profiles, ranking and matching
- 3.10 When things go wrong
- 3.11 Privacy, safety and fairness

Part 4. How we build it

- 4.1 Assumptions and golden rules
- 4.2 Who does what
- 4.3 Tools
- 4.4 Repo layout
- 4.5 Data formats
- 4.6 Feature list and priorities
- 4.7 Build roadmap
- 4.8 Hour-by-hour tasks
- 4.9 How we work as a team
- 4.10 Prompts for your AI coding tool

Part 5. The demo and the pitch

- 5.1 Demo hardware
- 5.2 Scenario packs
- 5.3 Making the fake data
- 5.4 The comparison simulation
- 5.5 The live demo, minute by minute
- 5.6 Lines to use in the pitch
- 5.7 Our trade-offs
- 5.8 Tough questions and our answers

Part 6. Who has tried this before

Part 7. Risks and the day itself

- 7.1 Risks and backups
- 7.2 Hackathon-day checklist

Glossary

Sources

PART 1

The problem

1.1 The hackathon in one minute

CODE//CONNECT'26 gives three problem statements from three unrelated areas: disaster response, threat prevention and misinformation control. Each team picks one and completes it. We picked **Problem 01: Disaster Response, "finding who needs help most"**.

Every team must follow five ground rules. Here is what each one means for us in practice:

Ground rule	What it means for us
Choose one problem statement and complete it.	We stay focused on Problem 01 and answer all three of its key questions, for every kind of disaster it mentions.
Deliver a working prototype that can be shown live, not only slides.	Real phones, a real laptop, real alerts and reports moving with the internet switched off. Slides only set up the demo.
Explain your trade-offs: what you chose, what you left out and why.	We keep an honest trade-offs list (section 5.7) and say it out loud in the pitch.
Be explicit about how your solution behaves when data is incomplete or wrong.	Confidence scores, duplicate merging, a fake-report guard, signed alerts, stale-report checks, Silent Zones for missing data, and a "chaos button" that shows all of this live.
Design for real users, real constraints and real contexts.	Built for dispatchers, volunteers and rescue teams; for the moment networks and power fail; and so people without smartphones still get warned and counted.

1.2 Problem 01 in plain words

When a flood, a heatwave or another disaster hits, warnings don't reach everyone. The people least likely to get a warning are often the ones in the most danger:

- elderly people
- people who live alone
- people in low-income neighbourhoods
- people with disabilities
- people without a smartphone or a reliable signal

At the same time, help is limited. There are only so many rescue teams, vehicles, shelter beds and medical supplies. So every decision about who to help first has a cost: helping one family means another family waits.

> **The core challenge:** use technology to find who needs help most, and get the limited resources to them *before* the situation becomes critical.

Which disasters? The problem statement names floods and heatwaves and says "and other disasters". So PulseTriage is not a flood app. It is one system with a small settings file for each kind of disaster: floods,

heatwaves, earthquakes, storms and cyclones, wildfires and landslides (section 2.3).

The three questions we must answer

Question	What the problem statement asks	Our short answer
Who is most at risk?	Risk depends on location, age, health, mobility, housing and access to information. Decide which signals show risk, and how to find people when warnings don't reach them.	Rules that read age, health, mobility and "alone" from every report; an opt-in registry of vulnerable people; warning coverage that shows who hasn't heard; Silent Zones that spot areas going quiet.
Who gets help first?	Teams, vehicles, shelters and supplies are limited. Match them to the greatest need, and justify the order in a way responders can trust.	Priority points based on time left, with an explanation card for every case; team and shelter suggestions that show who would wait longer.
How early can you act?	Help works best before things become critical. Think about early signals, forecasts, crowd reports and moving resources in advance.	Alerts that travel phone to phone before the worst arrives; ranking by hours left; "window closing" points; welfare checks for people who don't answer an alert.

Who is affected

- People in disaster-prone areas with no early warning.
- Elderly, disabled, chronically ill or isolated people who cannot get to safety on their own.
- Rescue coordinators and volunteers with too few vehicles and supplies.
- Shelter, cooling-centre and relief managers balancing space against demand.

Where it gets hard

- **Incomplete information:** reports come late, twice, wrong, or not at all from some areas.
- **Competing priorities:** a medical emergency, a stranded family and a blocked road all need the same team.
- **Connectivity failure:** networks and power often fail exactly when the system is needed.

1.3 The hidden trap: the quietest are in the most danger

> **A system that only ranks incoming messages ranks the people who could already reach us.** The loud get help first; the quiet wait. In almost every kind of disaster, the people in the most danger are the quietest.

Disaster	Why the people in most danger go quiet
Heatwave	Heat kills quietly. People alone indoors may not notice heatstroke until they are confused or collapse. In Chicago's 1995 heatwave, many of the hundreds who died were old, poor and living alone.
Earthquake	People trapped under rubble cannot call. Towers and power are often down across the whole area.

Disaster	Why the people in most danger go quiet
Flood	Towers and power fail; phones die; people trapped upstairs have no signal.
Storm or cyclone	Power and towers go down before landfall; people can't go outside to look for help.
Wildfire	Fire moves fast; people without a car or who can't walk are left behind while others leave.
Landslide	It happens in seconds, often in remote places with weak signal; roads are cut.

Most teams will build a system that reads messages and sorts them. That system has a blind spot exactly where the problem statement points. PulseTriage is built to look for silence too:

- **Who hasn't answered the warning?** Every alert has "I'm safe / I need help" buttons. Registered vulnerable people who don't answer become welfare checks.
- **Silent Zones:** if a hard-hit area with many vulnerable people goes quiet, that is a warning sign, not good news.
- **Helpers report for others:** the app is built for neighbours and volunteers to report the person next door who has no phone.
- **An opt-in registry:** people who need oxygen, dialysis or a wheelchair, or who live alone, can be known before the disaster, so their silence can be noticed.

1.4 Two gaps: warnings out, calls for help in

The problem has two halves, and we must close both:

Gap 1: the awareness gap

Warnings don't reach the people most at risk: no smartphone, no signal, a language they don't read well, low literacy, poor hearing or eyesight, or nobody around to tell them.

Our answer: alerts on every channel we can reach, including phone to phone without towers, read aloud, in local languages, and passed on by helpers knocking on doors. Then we check who hasn't heard (section 3.2).

Gap 2: the reporting gap

Calls for help don't reach responders, or arrive as a messy flood of duplicates, rumours and outdated messages.

Our answer: many ways in (app, alert replies, calls, radio, walk-ins, forwarded messages), all merged into one ranked list that explains itself, plus Silent Zones for those who never report (section 3.3).

1.5 What happens today

When a big disaster hits, help requests pour in through calls, texts, social media and shared spreadsheets. Under that overload, the order of rescue usually falls back to first-come-first-served, or to whoever is easiest to reach.

The clearest real example is Zhengzhou, China, in July 2021. During record floods, a university student started a shared online spreadsheet of people waiting for rescue. Volunteers sorted it by urgency on their own: pregnant women about to give birth, babies and trapped elderly people were marked "very urgent". Tencent reported over 2.5 million views and more than 20,000 edits. It helped save lives, but as it grew it

filled with duplicates, outdated rows and entries nobody had checked, and a professional team had to take it over.

Heatwaves show the other half of the problem. After Chicago's deadly 1995 heatwave, the city changed its approach: in a 1999 heatwave it issued strong warnings, kept cooling centres open and sent workers door to door to check on older residents, and far fewer people died.

> People already sort by vulnerability on their own, and knocking on the right doors saves lives. What they lack is a tool that warns everyone it can reach, notices who hasn't answered, merges the messy reports, says what it is unsure about, and keeps working when networks fail. That is PulseTriage.

PART 2

Our solution: PulseTriage

2.1 PulseTriage in one paragraph

PulseTriage is two connected apps that work for any kind of disaster. **PulseTriage Command** runs on one laptop at the command post. It sends warnings out, gathers every report that comes in, ranks every case by how much time is left and how vulnerable the people are, explains each ranking in plain words, spots people and areas that have gone suspiciously quiet, helps the dispatcher send the right team, and tracks every team and supply until the job is done. **PulseTriage Field** is an Android app carried by helpers, volunteers and rescue teams. It shows alerts and reads them aloud, records reports even with no signal, and passes both from phone to phone until they reach the people who need them. The whole system works with no internet.

> **Our one-line pitch:** "PulseTriage finds the people who can't call for help."

2.2 The loop: warn, hear back, rank, act

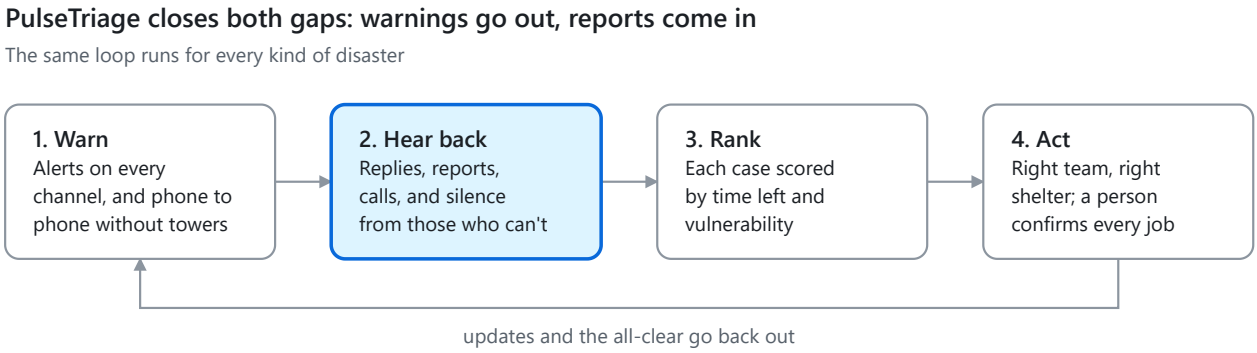


Figure 1. The PulseTriage loop. Step 2 is highlighted because "hearing back" includes noticing who has *not* answered.

1. **Warn.** The command post issues an alert for the affected area. It goes out on every channel we can reach: the Field app (phone to phone, even without towers), helpers knocking on doors, a short script for loudspeakers and radio, and lists of registered vulnerable people to visit or call.
2. **Hear back.** Replies to the alert ("I'm safe", "I need help"), reports from the app, phone calls, radio messages, walk-ins and forwarded messages all come in. So does silence: registered people who haven't answered, and areas sending far fewer reports than expected.
3. **Rank.** Every case gets priority points based on how soon it becomes life-threatening and how unable people are to save themselves, adjusted for how much we trust the report. Every point is shown.
4. **Act.** The dashboard suggests the fastest team that can actually help, and the nearest shelter or cooling centre with space, and shows who would wait longer as a result. A person confirms every job. Updates and the all-clear go back out the same way.

2.3 One system for every disaster

The engine never changes. What changes from one disaster to another lives in a small settings file called a **hazard profile**: which questions the app asks, how fast each situation becomes critical, which kind of team is needed, what "window closing" means, which area counts as affected, the safety tips, and the alert wording. Adding a new disaster type means writing one new profile, not new code.

Disaster	What usually hurts people	What the app asks	Who gets sent	"Window closing" means	Affected area
Flood	Drowning; being cut off; medical needs with no access	Water level, rising?, trapped?	Boat (waist-deep or more), medic	Their road is about to flood	Inside the flood extent
Heatwave	Heatstroke, especially elderly, infants, the ill and people alone	Very hot indoors? Fan, AC or cool place? Signs of heat illness? Power cut?	Medic; cooling and welfare team; transport to a cooling centre	The hottest hours are coming and they have no cooling	The heat-alert area
Earthquake	Collapse; being trapped; fire and gas	Trapped? Building damage? Gas or fire? Injuries?	Search and rescue, medic	Night is falling or rain is coming; chances drop every hour someone is trapped	The strong-shaking area
Storm or cyclone	Flying debris; collapse; storm-surge flooding	Roof or wall damage? Surge water? Need shelter?	Truck before landfall; boat after the surge	Landfall: teams must stop when the wind is too strong	The storm's path
Wildfire	Being trapped by fire; smoke	How close is the fire? Smoke? Route blocked? Can you drive?	Evacuation vehicle, fire crew, medic	The fire front is about to reach their only road	The evacuation zone
Landslide	Being buried; house collapse	Buried? Cracks? Slope moving? Road blocked?	Search and rescue, truck	More heavy rain is forecast	Near the slide area

These are general design notes, not expert guidance. Before real use, every profile should be checked with local emergency services and health authorities.

What we build in the hackathon: the profile engine plus fully tested **flood** and **heatwave** profiles (Must), an **earthquake** profile (Should), and **storm, wildfire and landslide** profiles as settings files (Stretch). The demo runs a flood, then switches to a heatwave in 20 seconds to prove the same engine handles both.

2.4 Where the idea came from

We started with three ideas. Each had a strong part and a weak part. PulseTriage keeps the strong parts and fixes the weak ones.

Concept	The idea	Strong part	Weak part	What we kept
1. PulseTriage	Read messy messages, rank people by risk, explain why, keep working offline.	Explainable ranking; easy to demo.	Only sees people who send messages; didn't warn anyone.	**The brain:** ranking, explanation cards, merging, confidence.
2. BeaconRelief	Phones pass distress messages to each other with no internet; neighbours help neighbours.	Survives network failure; uses nearby helpers.	A web app can't talk phone to phone; assumes victims have smartphones.	**The delivery layer, both ways:** alerts out and reports in over an Android mesh; helpers reporting and warning for others.
3. PrePosition AI	Forecast what will be cut off, to move resources early.	Acts before things become critical.	Needs lots of data; risky to fake in 36 hours.	**Acting early:** early alerts, and "window closing" points that move cases up while help can still get in.

Why a mobile app and not a website? A website can already work offline, but a browser cannot send data directly from one phone to another. A native Android app can, using Google's Nearby Connections (Bluetooth and Wi-Fi Direct, no internet or SIM card needed). That is what lets both alerts and reports travel when the towers are down.

2.5 The two apps

PulseTriage Field (Android)

Who uses it: helpers (neighbours, residents), volunteers (people with a boat, a 4x4, medical skills) and rescue teams.

What it does:

- Shows alerts full screen with sound, reads them aloud, and points to the nearest shelter or cooling centre.
- "I'm safe / I need help / I'm helping others" in one tap.
- Reminds helpers to pass the warning on to neighbours.
- Report for yourself or someone else in about 20 seconds; the questions change with the disaster.
- Works with no signal; passes alerts and reports phone to phone.
- With Wi-Fi or mobile data that reaches the server, sends directly; only with no connection does it pass messages phone to phone.
- Receives rescue jobs and welfare checks; QR backup.

Built with Flutter, Android only.

PulseTriage Command (laptop)

Who uses it: dispatchers and call-takers at the command post (a school, a town hall, a tent).

What it does:

- Writes and sends alerts from templates, for any zone and disaster type.
- Warning coverage: who has and hasn't answered.
- Takes reports from calls, radio and walk-ins, and from pasted messages.
- Ranked list of every case, with an explanation card for each.
- Map of cases, teams, zones, shelters and cooling centres.
- Silent Zones painted red, with a "send welfare check" button.
- Team and shelter suggestions that show who would wait longer.
- Tracks every team (free, on the way, arrived) and where it is, plus the supplies left.

A FastAPI server and a React dashboard on one laptop, with its own Wi-Fi router. No internet needed.

2.6 The seven ideas that make it different

1. **Warnings that travel without towers, and know who hasn't heard.** Alerts hop phone to phone, are read aloud, and come with one-tap replies. The dashboard shows, area by area, which registered vulnerable people haven't answered, and turns them into welfare checks.
2. **Silence is a signal (Silent Zones).** For every hard-hit area, we compare how many reports it *should* produce with how many it does. An area that goes quiet gets a welfare-check team. Nobody has to ask first.
3. **Helpers warn and report for people who can't.** The app is built for neighbours, volunteers and teams, not for victims to install in advance. Someone with no phone still gets warned when a helper knocks, and still gets counted when the person next door reports them.
4. **One engine, every disaster.** A hazard profile per disaster type changes the questions, the time rules, the teams and the alert wording. The ranking, mesh and dashboard stay the same.

5. **Ranked by time left, explained in plain words.** The AI only *reads* messages; simple, visible rules do the ranking, so responders can trust it and authorities can tune it.
6. **Honest about uncertainty.** Every case has a confidence score. Duplicates merge. Big claims from unknown phones get a "verify" badge. Old reports get flagged for a re-check. Low confidence moves a case down but never hides it.
7. **Works with no internet.** Phones send directly when they can reach the server and relay to each other only when they can't; the command post runs on a laptop and its own router; if the AI service can't be reached, keyword rules take over. A QR code moves messages if Bluetooth fails.

2.7 How we answer every part of the problem statement

The problem statement says	How PulseTriage answers	Feature
Warnings may not reach everyone.	Alerts on every channel we can reach, including phone to phone without towers, read aloud, in local languages, passed on door to door; then we track who hasn't answered.	Alerts, warning coverage
Floods, heatwaves and other disasters.	One engine with a hazard profile per disaster type.	Hazard profiles
Elderly, disabled, ill or isolated people can't get to safety alone.	Extra points for "can't save themselves"; registry; neighbours can report for them.	Priority points, registry, "report for a neighbour"
People without smartphones or signal.	Helpers warn and report for them; the mesh carries messages without signal; calls and walk-ins go in through the call-taker form; loudspeaker scripts.	Field app, mesh, intake form, announcements
Rescue teams, vehicles, shelters and supplies are finite.	Suggest the fastest free team that can help and the nearest shelter with space; show who waits longer; track every team's status, location and arrival, and the supplies left.	Team assignment, team tracking, supplies
Shelter and relief managers balance capacity against demand.	Shelter head counts and supply stock, updated from outcome forms and calls; low-stock warnings and a burn-rate forecast.	Shelters, supply inventory
Justify the order so responders trust it.	Every point is visible; a dispatcher can override, with a reason that is logged.	Explanation card, override log
Act early, before it becomes critical.	Early alerts; rank by hours left; "window closing" points; welfare checks for people who don't answer.	Alerts, hours to critical, window closing
Reports are late, duplicated, inaccurate or missing.	Merge duplicates, score confidence, re-check stale reports, guard against fakes, detect silence.	Merging, confidence, Silent Zones
Competing priorities.	One points system for every kind of case and disaster, including zone checks.	Priority points
Networks and power fail.	Phone-to-phone relay, a laptop command post, keyword backup, QR backup.	Mesh, offline mode

2.8 What we are deliberately not building

Saying no is part of the plan. These are our conscious choices, and they go on the trade-offs slide:

- **Replacing official warning systems.** Governments own cell broadcast alerts (like FR-Alert in France or Japan's earthquake warnings). We complement them for the places, people and moments they miss; we would connect to them, never compete with them.
- **Fully tested profiles for every disaster.** We fully test flood and heatwave, add earthquake if time allows, and include the others as settings files.
- **iPhone support.** A mesh that mixes Android and iPhone is hard. Android only for the hackathon.
- **A model that learns its own weights.** We have no real training data, and learned weights are hard to explain.
- **Live hazard feeds and real road routing.** Our scenarios script the flood extent, heat hours and road closures. Live feeds and OpenStreetMap routing are stretch goals.
- **Mass text messages and automated phone calls.** They need a paid service and a working network. Stretch goals.
- **Automatic dispatch.** A person always confirms. Software suggests; humans decide.

PART 3

How it works

3.1 The big picture

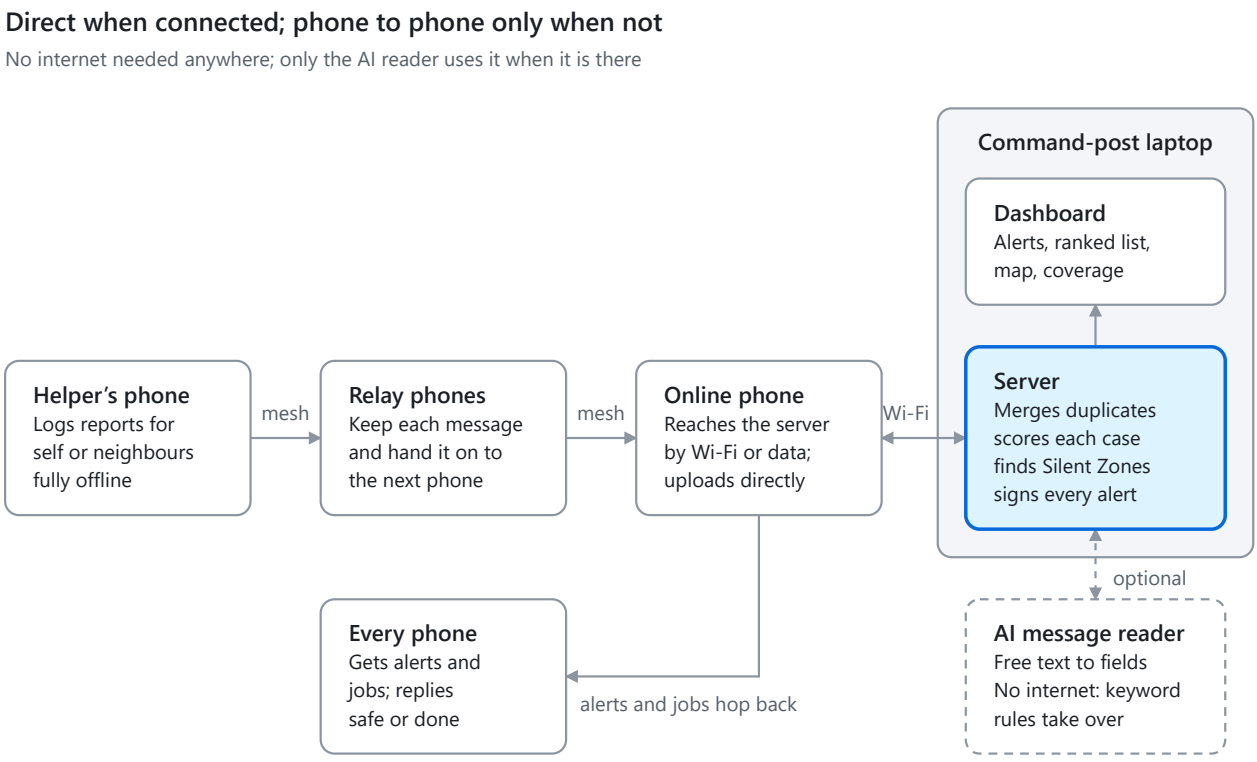


Figure 2. The whole system. A phone that can reach the server (Wi-Fi or mobile data) uploads directly. Only phones with no connection relay over Bluetooth and Wi-Fi Direct until a message reaches an online phone. Alerts and jobs travel out the same way reports travel in.

There are five parts:

1. **Helper's phone:** anyone with the Field app can log a report for themselves or for someone else, and reply to alerts. Everything is saved on the phone instantly; with a connection it goes straight to the server.
2. **Relay phones:** a phone with no connection carries copies of the alerts and reports it has seen and hands them to the next phone it meets. Messages spread like a note passed along a line of people.
3. **Online phone:** any phone that can reach the server (over the command-post Wi-Fi, home Wi-Fi or mobile data) sends directly and fetches alerts and jobs. It also uploads whatever offline neighbours hand it. It happens automatically.
4. **Command-post laptop:** the server receives reports, merges duplicates, scores every case, tracks warning coverage, teams and supplies, finds Silent Zones and signs every alert. The dashboard shows it all to the dispatcher.
5. **AI message reader (optional):** turns free-text messages into clear fields when the internet is available. Without internet, keyword rules do the same job, a little less well.

3.2 Warning people: how alerts reach everyone

The problem statement starts with "warnings may not reach everyone". This is how PulseTriage tries to reach everyone it can, and then finds out who it missed.

Step 1. The dispatcher writes an alert in about 30 seconds

On the dashboard, "New alert" opens a form with a template for the chosen disaster. The dispatcher picks:

- **Disaster type** (flood, heat, earthquake, storm, wildfire, landslide).
- **Severity:** *Watch* (be ready), *Warning* (act today) or *Emergency* (act now).
- **Area:** one or more zones on the map.
- **Headline and up to three "do this now" steps,** pre-filled from the hazard profile and editable.
- **Places to go:** the nearest shelters or cooling centres with space, filled in automatically.
- **Languages:** the local languages to include. Templates are stored for each language; with internet, the AI can help translate new wording.
- **Valid until:** when the alert expires. Updates and the all-clear are sent the same way.

The server **signs** every alert with the command post's private key, so phones can tell a real alert from a fake one.

Step 2. The alert goes out on every channel at once

Channel	Who it reaches	How	In our build
Field app, phone to phone	Everyone with the app, even with no towers	Alerts are the highest-priority packets in the mesh: always sent first	Must
Helpers knocking on doors	Neighbours with no phone, no signal, or who can't read	Every alert on the app asks helpers to pass it on: "Check on neighbours who are elderly, ill, alone or have small children"	Must
Loudspeaker and radio script	Anyone within earshot, including people who can't read	The dashboard writes a 20-second announcement in the chosen languages, for community centres, places of worship, megaphones and local radio	Should
Outreach list	Registered vulnerable people in the area	A list (on volunteers' phones or printed) of who to visit or call first, highest risk first	Should
Text messages	Registered people with any mobile phone	An Android phone at the command post sends texts when the network allows	Stretch
Automated voice calls	Registered people with any phone, including landlines	"Press 1 if you are safe, 2 if you need help"	Stretch

Channel	Who it reaches	How	In our build
Official cell broadcast	Every phone in range of a working tower	Run by governments (for example FR-Alert in France, earthquake warnings in Japan). We'd connect to it, never replace it	Not ours

Step 3. On the phone: impossible to miss, easy to understand

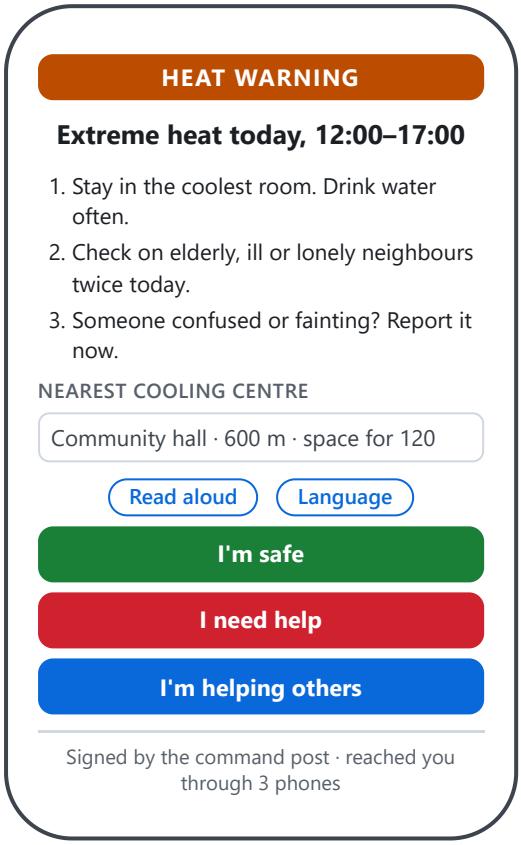


Figure 3. Alert screen.

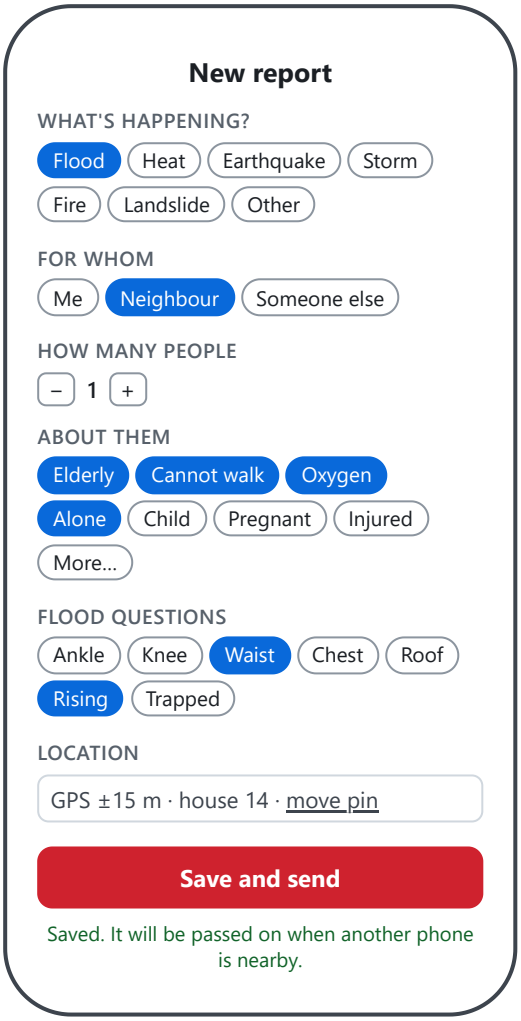


Figure 4. Report form. The last questions change with the disaster.

- **Full screen with sound and vibration,** a colour band per severity and a big icon for the disaster type.
- **Read aloud** with the phone's built-in text-to-speech, which works offline. Helps people who can't read well or see well.
- **Three short steps, plain words,** in the chosen language, plus the nearest shelter or cooling centre.
- **Three reply buttons:** "I'm safe", "I need help" (opens the report form, already filled in), "I'm helping others".
- **"Pass it on"** reminder for helpers, and offline safety tips for that disaster from the hazard profile.
- **Only real alerts show:** the phone checks the command post's signature using the key it saved at setup. Anything unsigned is ignored, so nobody can start a panic with a fake alert.

Step 4. Who hasn't heard? Warning coverage

Sending a warning isn't the goal; people knowing about it is. For each zone, the dashboard counts how many registered vulnerable people are **confirmed**: they replied to the alert, a helper did a welfare check on them, or a call-taker reached them.

- **Warning coverage** = confirmed people ÷ registered people in the zone. It shows as a bar per zone and a colour on the map.
- For an **Emergency**, anyone still unconfirmed after 30 minutes becomes a welfare-check case. For a **Warning**, after 2 hours.
- People who need oxygen, dialysis or insulin, or who live alone, go to the top of the outreach list.
- A zone with coverage below 25% that is also hard-hit feeds straight into the Silent Zone check (section 3.9).

> This is the same idea as Chicago's door-to-door checks in 1999 and Bangladesh's cyclone volunteers with megaphones: technology tells the volunteers *which* doors to knock on first.

3.3 Hearing back: every way a report comes in

Every channel feeds the same pipeline: read, merge, score, rank. Nobody has to pick "the right" channel.

Channel	Who uses it	How it reaches the server	In our build
Field app report form	Helpers, volunteers, teams	Directly when online; through the mesh when not	Must
Alert replies	Anyone with the app	Mesh; "I need help" opens the form	Must
Call-taker form	Operators taking phone calls, radio messages and people who walk in	Typed straight into the dashboard	Must
Welfare-check results	Helpers and volunteers who visited someone	Mesh: "Checked on house 14: safe / needs help"	Should
Paste box	Volunteers copying messages from WhatsApp, social media or shared spreadsheets	Pasted into the dashboard; the AI reader splits and reads them	Should
Text messages to a hotline number	Anyone with any mobile phone	An Android phone at the command post forwards incoming texts	Stretch
Silence	Nobody: it is the absence of a reply	Unanswered alerts and quiet zones become cases automatically	Should

3.4 Two journeys: a flood and a heatwave

These two stories show the same system handling two very different disasters. Every teammate should be able to tell them.

Journey 1: a flood (someone reports for a neighbour)

1. **14:10. The warning.** The command post issues a flood warning for Ward 4 and Riverside. Sam's phone shows it full screen and reads it aloud. Sam taps "I'm helping others".
2. **15:05. The report.** Water is rising in Ward 4 and the phone towers are down. Sam checks on the old man next door in house 14: he can't walk, he lives alone, and his oxygen tank is half empty. Sam opens the Field app, picks Flood and "Neighbour", taps Elderly, Cannot walk, Oxygen and Alone, picks "Waist" and "Rising", and taps Send. It takes about 20 seconds and saves with no signal.
3. **15:12. The first hop.** A volunteer passes by. Their phones find each other over Bluetooth and Sam's report copies onto the volunteer's phone. Nobody presses anything.
4. **15:30. Online.** The volunteer reaches the school used as the command post. Their phone joins the command-post Wi-Fi, goes online and uploads everything it carries.
5. **15:30. The server.** It opens the sealed report, checks its signature, and merges it with one other report about house 14 from another neighbour's phone ("2 reports"). Confidence 0.75. Points: 69.
6. **15:31. The dispatcher.** The explanation card says why: oxygen with about 3 hours left (+40), cannot walk (+15), elderly (+10), alone (+10), 1 person (+2), times 0.9 for trust. Ward 4 had been red as a Silent Zone; now reports are arriving and the red fades.
7. **15:32. The decision.** Water is waist-deep and he needs oxygen, so it must be a boat with a medic: Boat 2, about 13 minutes away. The screen warns: "Boat 2 is also best for case #3. If it goes here, #3 waits about 35 more minutes (next best: Boat 4)." The dispatcher confirms.
8. **15:33 to 16:05. The loop back.** The job hops to Boat 2's phone; "On my way" hops back and Sam's phone shows "Help is coming". Boat 2's location pings move its icon across the map; at 15:47 a ping lands within 100 m of house 14 and the dashboard marks it "on scene". At 16:05 Boat 2 taps "Done": 1 person taken to the school shelter, 1 oxygen cylinder used. The shelter's head count and the oxygen stock update. No internet was needed at any point.

Journey 2: a heatwave (nobody calls at all)

1. **07:00. The warning.** The command post issues a heat warning for Ward 4 and Old Market: extreme heat from 12:00 to 17:00. It reaches phones through the mesh; a volunteer reads the 20-second announcement at the community centre; 41 registered vulnerable people live in Ward 4.
2. **09:00. Coverage.** 32 of the 41 are confirmed: some tapped "I'm safe", some were called, some were visited. Coverage 78%. Nine are not.
3. **09:00. Silence becomes cases.** It's a Warning, so after 2 hours the nine unconfirmed people become welfare-check cases. One is an 81-year-old woman who lives alone on a top floor, has no fan and no smartphone. She is registered as "lives alone".
4. **10:15. The knock.** A nearby volunteer gets the welfare check on their phone (through the mesh). She is confused and very hot. The volunteer taps "Needs help", picks Heat, and taps "Confused" and "Hot, dry skin".
5. **10:16. The server.** Heatstroke signs mean about 30 minutes left: +50. Elderly +10, alone +10, 1 person +2: 72. The volunteer is registered and GPS is good: confidence 0.8, times 0.92: **66 points**. Needs: a medic.
6. **10:17. The decision.** The dashboard suggests Medic 1, 9 minutes away, and the cooling centre with space. The volunteer's phone shows the heat first-aid tips from the profile (written by health authorities) while they wait.

7. **10:40. Done.** The medic team takes her to care. **Nobody in this story called for help.** The system noticed the silence and sent someone to knock.

3.5 The Field app

Three roles

Each person picks a role on first launch. Any phone that can reach the server sends directly; only phones with no connection use phone-to-phone relay.

Role	Who	What they can do
Helper	Residents and neighbours	Get alerts and reply; pass warnings on; report for themselves or others; do welfare checks; relay messages.
Volunteer	People with a boat, a 4x4 or medical skills	Everything a helper does, plus share their resources, get outreach lists, and accept jobs and welfare checks.
Rescue team	Official teams	Receive jobs; send location pings; mark jobs "on my way", "arrived" or "done" with an outcome. Their reports start with more trust.

Seven screens

1. **Setup** (first launch only): pick a role, an optional name, languages and skills (boat, 4x4, nurse or doctor, first aid). Scan the setup QR code on the dashboard; it saves the server address and the command post's public key, which the phone uses to check alerts. The phone creates its own signing key.
2. **Home:** the latest alert at the top, one big red "Report someone who needs help" button, and a status line: "3 phones nearby · 5 messages waiting · last sync 2 min ago".
3. **Alert** (Figure 3): full screen, read aloud, three steps, nearest shelter or cooling centre, three reply buttons, "pass it on" and safety tips.
4. **Report form** (Figure 4): "What's happening?" first (pre-set from the latest alert), then for whom, how many people, who they are, the questions from that disaster's profile, a note, and GPS with "move the pin". Send saves it in under a second, even with no signal.
5. **Welfare check:** "I checked on someone": who or where, then Safe or Needs help. Volunteers also see their outreach list here.
6. **Jobs** (volunteers and rescue teams): jobs addressed to this phone, with distance, direction, conditions and the profile's first-aid tips. Buttons: On my way, Arrived (also automatic within 100 m), Done (opens a 10-second outcome form) and Can't reach. Team phones also send location pings and have Out of service and Ready buttons and a fuel picker.
7. **Mesh and QR:** phones nearby, messages carried, Show QR and Scan QR, and a relay on/off switch we use during the demo.

3.6 The Command dashboard

One screen designed to be read from across a room (and on a projector). It refreshes every 2 seconds.

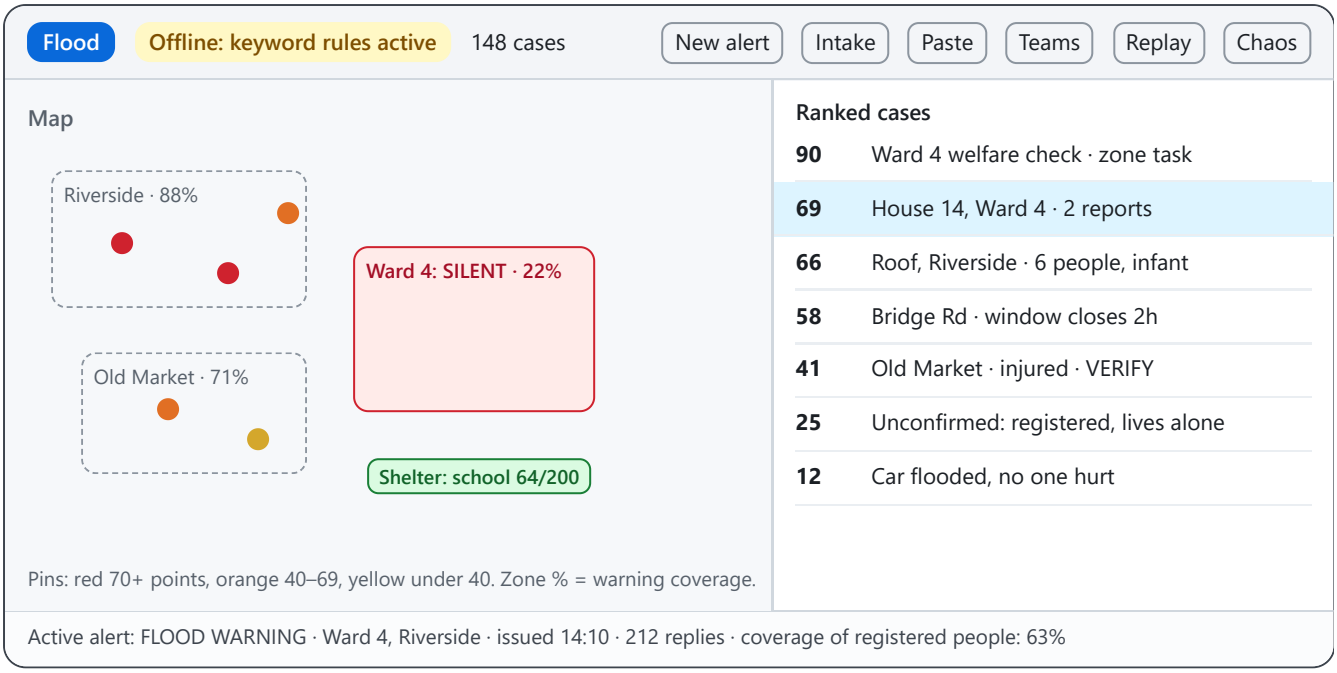


Figure 5. Dashboard layout: top bar with the disaster type, map with warning coverage per zone, ranked list, and the active alert along the bottom.

#2 · House 14, Ward 4 · Flood **69**

Oxygen, about 3 h left (time band 1–3 h)	+40
Cannot walk	+15
Elderly	+10
Lives alone	+10
1 person	+2
Subtotal	77
Trust 0.75: 2 reports from 2 phones, GPS ±15 m	×0.90
Priority points	**69**

2 reports · Needs: boat + medic · Read by: app form

Verified · Override… · Suggest team

Figure 6. Explanation card: the real maths, line by line.

Ward 4 · Silent Zone **90**

Hard-hit since 14:40. **Expected about 24 reports, got 2.**

Warning coverage: **9 of 41** registered vulnerable people confirmed (22%).

6 of the unconfirmed need oxygen or dialysis. Possible phone outage.

Zone task: 60 points + 1 per registered vulnerable person, capped at 90.

Send welfare-check team · Outreach list

Figure 7. Silent Zone card, with warning coverage.

Dashboard parts

- **Top bar:** the active disaster type (switches the hazard profile), the offline banner (only when the AI reader is unreachable), counts, and buttons for New alert, Intake, Paste, Teams, Replay and Chaos.
- **Alert composer:** template per disaster and severity, zone picker, editable steps, places to go, languages, preview of the phone screen, and "Generate announcement script".
- **Map:** case pins coloured by points, zones coloured by warning coverage (Silent Zones filled red), teams, shelters and cooling centres with space left, and window-closing markers.
- **Ranked list:** every open case, highest points first, with badges (verify, re-check, unconfirmed, number of reports).

- **Explanation card:** each reason with its points, the trust line, badges, and buttons: Verified, Override (a reason is required and logged), Suggest team, Split (undo a wrong merge).
- **Intake form:** for call-takers: a quick form with the same fields as the phone, plus free text for the AI reader.
- **Paste box:** paste a batch of forwarded messages; the AI reader splits and reads them into cases.
- **Teams and resources panel** (section 3.7): counters, the team table with status, ETA and last seen, supplies left, shelters' head counts and warnings; Suggest and Confirm; the trade-off message.
- **Weights panel:** a slider per weight so a local authority can tune the rules. The list re-ranks instantly.
- **Chaos panel (demo only):** inject a duplicate, a fake report, a wrong location, or an AI outage.

3.7 Tracking teams and resources

The dashboard always shows which teams are free, where each team is, whether it has arrived, and how much of each supply is left. That is what lets a dispatcher trust the suggestions, and notice when a team is in trouble.

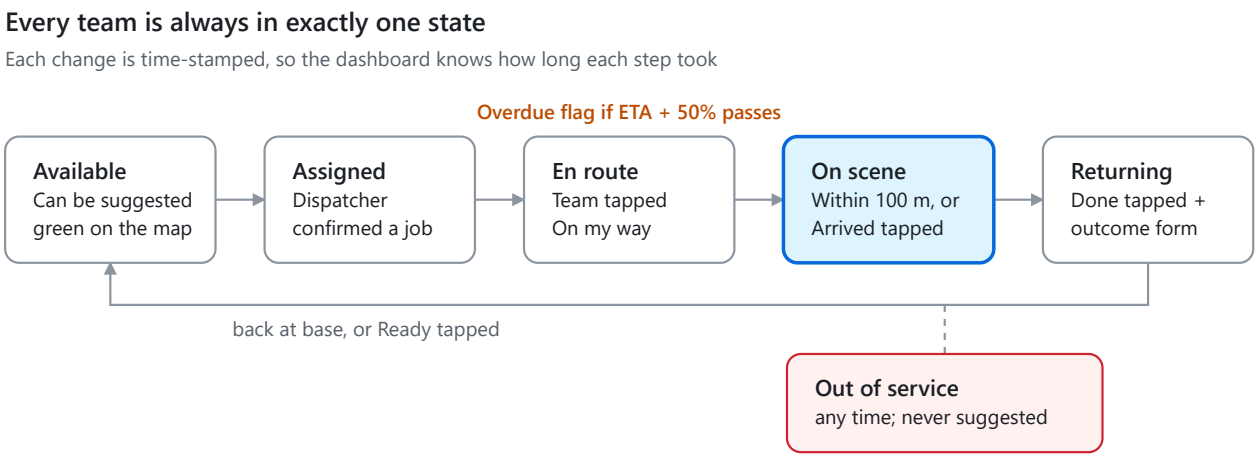


Figure 8. Team states. "On scene" is set automatically when a team's location ping lands within 100 m of the case, or when the team taps "Arrived".

Team states

Status	Set by	On the dashboard
Available	Team phone ("Ready") or dispatcher	Green icon at its last position; can be suggested
Assigned	Dispatcher confirms a job	Blue; job and ETA shown; not suggested for other cases
En route	Team taps "On my way"	Blue icon with a line to the case and the ETA counting down
On scene	Ping within 100 m of the case, or "Arrived" tapped	Purple; arrival time logged; the case shows "on scene"
Returning	Team taps "Done" and sends the outcome form	Grey-blue; Available again at base or when "Ready" is tapped
Out of service	Team or dispatcher, with a reason (broken, refuelling, crew resting)	Red; never suggested until cleared

Live team locations

- Rescue-team and volunteer phones send a small **location ping**: every 2 minutes on a job, every 10 minutes when idle. A ping carries position, accuracy, status, battery and fuel level (full, half, low).
- Pings travel through the mesh like any packet, sealed to the command post. Phones keep only the **newest ping per team** and drop older ones, so pings never clog the mesh.
- Each team shows at its last known position with the time of that position ("as of 15:41, received 15:44"). The icon turns **grey** when the position is over 10 minutes old.
- A short trail of the last 5 positions shows which way a team is moving.
- In the demo, the replay moves the scenario teams with simulated pings; Phone B on stage uses the "Arrived" button.

Did they get there?

- **Arrival** is confirmed automatically when a ping lands within 100 m of the case, or when the team taps "Arrived". Both together count as confirmed.
- **Overdue:** if a team hasn't arrived by its ETA plus 50% (at least 10 minutes), the dashboard flags it: "Boat 3 overdue: ETA 15:45, last seen 15:40, 1.2 km away". The dispatcher can call or radio, reassign, or send another team.
- **Not heard from:** no ping for 15 minutes while en route also raises a flag.
- **Outcome form:** on "Done", the team fills a 10-second form: people helped, where they were taken (shelter, cooling centre, hospital, stayed), supplies used, or "can't reach" with a reason. It updates the case, the shelter's head count and the supply stock.

Supplies left and used

- One inventory covers the command post, each shelter or cooling centre, and each team's load: drinking water, food packs, first-aid kits, oxygen cylinders, insulin and other medicines, blankets, life jackets, fuel and tarpaulins. Each profile suggests its own items (for example oral rehydration salts for heat).
- Every movement is logged: restock, load onto a team, delivered, used, transferred. **Left = stock − loaded − used.**
- **Low-stock warning** at a set level per item: "Oxygen cylinders: 3 left (warn at 5)".
- **Burn rate** (Should): use over the last 3 hours predicts "Drinking water runs out in about 9 h at the current rate".
- Shelter managers or call-takers update shelter stock and head count by phone or on the dashboard.

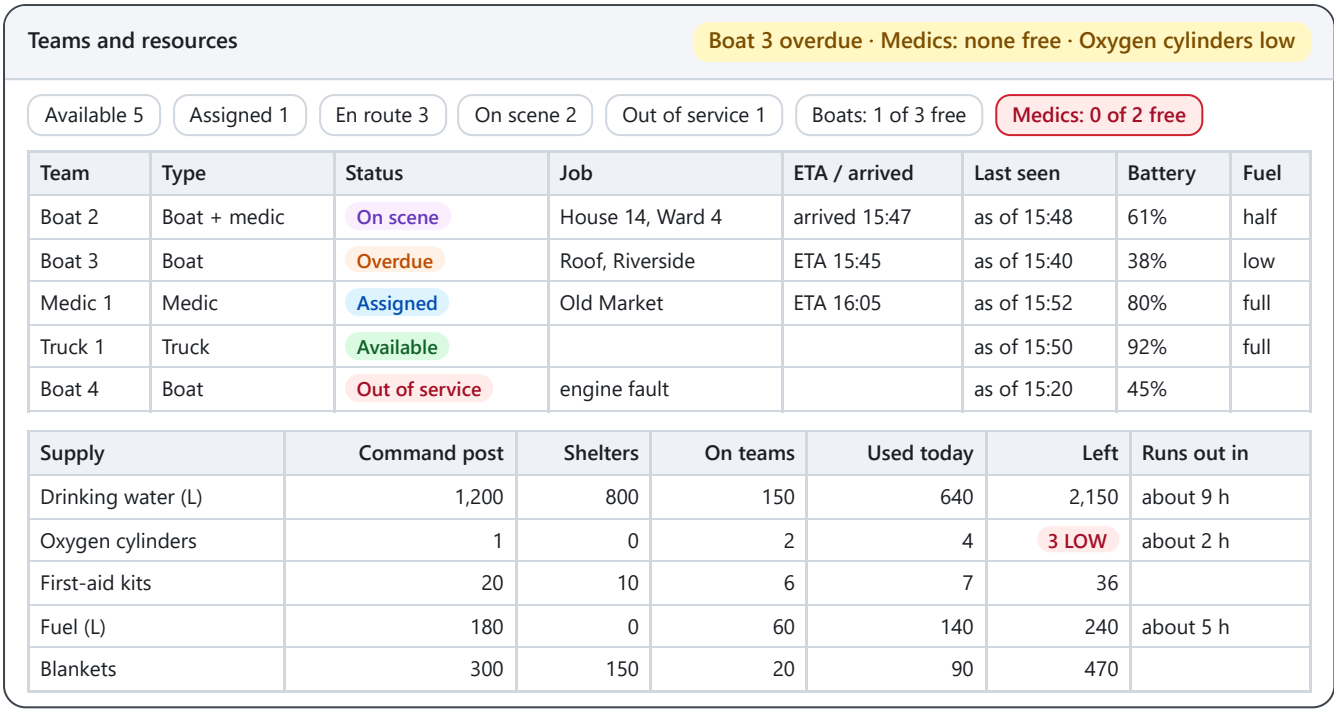


Figure 9. The Teams and resources panel: counters, warnings, every team's status and last position, and supplies left.

How it feeds the suggestions

- Only Available teams with fuel not "low" are suggested.
- Teams close to finishing a job are offered as "free in about 10 min".
- A job that needs oxygen or insulin goes to a team carrying it, or the suggestion says where to pick it up first.
- Counters turn red when no team of a needed type is free, so the dispatcher can ask for more help early.

3.8 How phones pass messages to each other (the mesh)

The simple idea. A normal message needs a phone tower. Ours doesn't. Each phone keeps a copy of every alert and report it has seen and hands copies to every phone it meets. People move, so messages move with them, hop by hop. This is called **store and forward**, and networks built this way are called **delay-tolerant networks**. Reports flow towards the command post; alerts and jobs flow away from it.

Direct first, mesh only as a fallback. A phone that can reach the server, over the command-post Wi-Fi, home Wi-Fi or mobile data, sends everything straight to it. Only a phone that can't reach the server uses the mesh. The phone checks on its own and switches automatically.

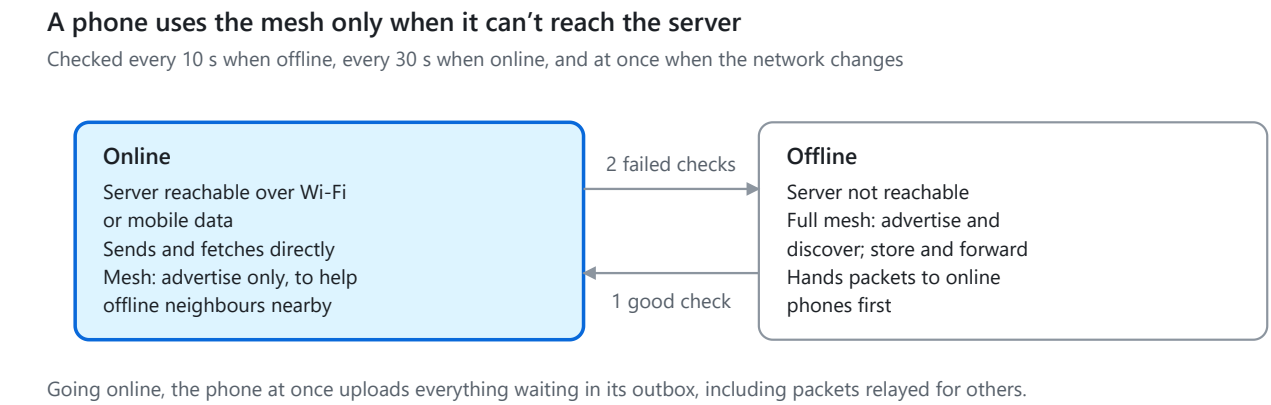


Figure 10. Connection modes. "Reach the server" means the server actually answers, not just "Wi-Fi is on". An online phone still listens for offline neighbours and uploads what they hand it (the "Help offline neighbours" setting, on by default).

The envelope. Each message travels as a "packet", which works like an envelope:

- **The outside** (anyone can read it): a unique ID, the kind of message, when it was made and expires, how many hops it has made, a rough area, and an urgency hint so phones pass on the most urgent messages first.
- **The inside of a report** (sealed): names, health details and the exact location. Only the command post can open it, so strangers' phones can carry it but cannot read it.
- **Alerts are not sealed** (everyone should read them) but they are **signed by the command post**, so they can't be faked.
- **The signature** (like a wax seal): proves who made a message and that nobody changed it on the way.

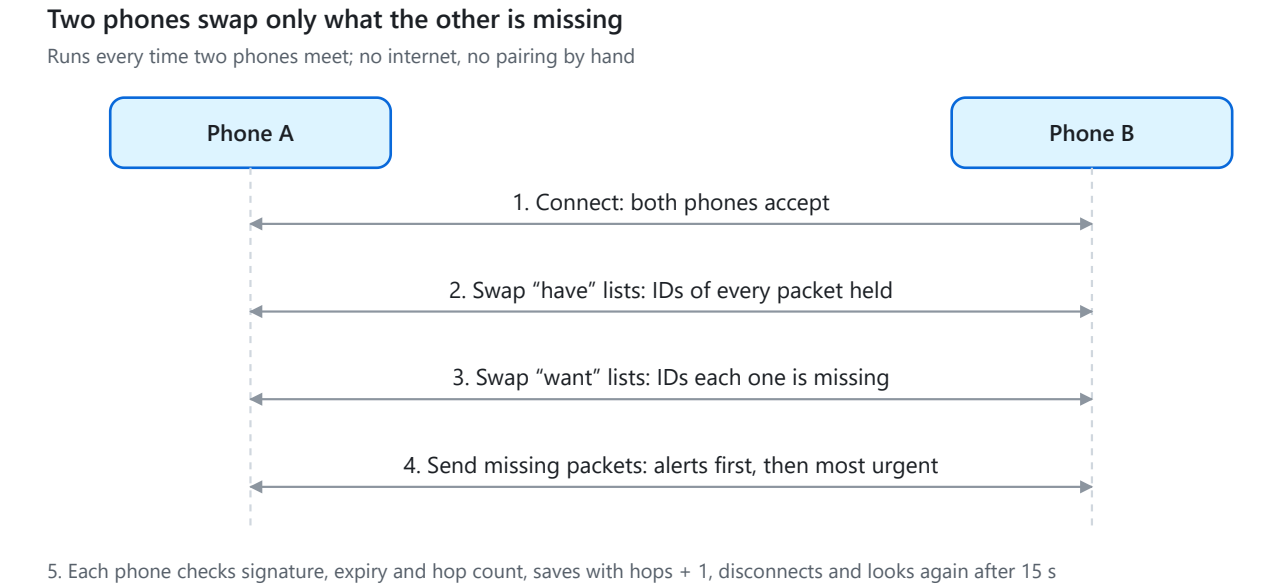


Figure 11. What happens when two phones meet.

The mesh loop, step by step

1. When the phone can't reach the server, it starts **advertising** ("I'm here") and **discovering** ("who's around?") at the same time, using Nearby Connections with the strategy `P2P_CLUSTER` (many phones to many phones) and the service ID `pulsetriage`. An online phone only advertises, with "on" in its name, so offline phones connect to it first.

2. When it finds a phone, it asks to connect. For the demo both sides accept automatically. (In real life both phones would show a 4-character code to compare.)
3. Once connected, each phone sends a **"have" list**: the IDs of every packet it holds.
4. Each side replies with a **"want" list**: the IDs it is missing.
5. Each side sends the missing packets: **alerts first** (urgency 4), then reports by urgency (3 down to 0), newest first, in small batches well under 32 KB each.
6. The receiver checks each packet: valid signature (alerts must carry the command post's), not expired, hop count below the limit (12), not already stored. Then it saves it with the hop count increased by 1. A new alert pops up full screen.
7. After the swap, the phones disconnect, wait 15 seconds and look again, so they don't get stuck on one partner.
8. Every 10 seconds, and at once when Wi-Fi or mobile data comes back, the phone tries to reach the server. If it answers, the phone goes **online**: it stops discovering, uploads everything not yet delivered, and from then on sends and fetches directly. After 2 failed checks in a row it goes back to the mesh.
9. Jobs travel out the same way. A phone shows a job only if it is addressed to that phone; otherwise it just carries it on.

QR backup: if Bluetooth won't connect, "Show QR" packs up to about 10 waiting packets into one QR code (compressed); with more, it cycles through several codes. "Scan QR" on another phone stores them exactly as if they arrived over the mesh. It works with any phone camera and is our on-stage backup.

3.9 The brain: hazard profiles, ranking and matching

> **The rule we never break:** the AI only *reads* messages. All ranking is plain rules with visible points. That keeps every decision explainable, lets authorities tune it, and lets everything run with no internet. Every number below is a **demo default**, not a medical or engineering fact. We say so in the pitch, and we would tune them with real responders.

a. Reading messages: the AI reader and its offline backup

1. Reports from the app form and the intake form are already structured, so they skip this step.
2. For free text (a note, a pasted message, a text, a scenario message), the server sends it to the AI (Claude, model `claude-haiku-5-5`) with a fixed instruction: say which disaster it is about, then reply only with the fields of that disaster's profile plus people, conditions, landmark and a confidence from 0 to 1; never invent details; unknown means empty.
3. If there is no internet, or the reply takes over 4 seconds, the **keyword reader** runs instead. Each profile carries its own word lists ("water", "roof", "boat" for floods; "heat", "fainted", "no fan" for heat; "trapped", "rubble", "gas" for earthquakes), plus shared lists for people ("oxygen", "wheelchair", "baby", "pregnant", "grandmother"...). We add the same words in our local languages.
4. If the disaster type is unclear, the reader uses the active alert's disaster type.
5. The case remembers which reader was used, so the card can say "read by keyword rules (AI offline)".

b. Hazard profiles: one settings file per disaster

Each profile is a small JSON file in `shared/profiles/`. It contains:

- **Questions** the app and the intake form ask for that disaster.
- **Hours rules:** which answers mean how many hours are left.
- **Needs rules:** which kind of team each situation needs.
- **Window rule:** what "window closing" means for that disaster.
- **Affected area:** which zones count as hard-hit (flood extent, heat-alert area, shaking area...).
- **Keywords** for the offline reader, **safety tips** and **first-aid tips** (wording from health and disaster authorities), and **alert and announcement templates** per severity and language.

A shortened example of the heatwave profile is in section 4.5.

c. Hours to critical

For each case, we estimate how many hours are left before it becomes life-threatening, and take the **smallest** value that applies. The first block applies to every disaster; then each profile adds its own rules.

Signal in the report	Hours left (demo default)
Any disaster	
Unconscious, or heavy bleeding	0.5
Needs oxygen	3 (or what the note says)
Medical device needs power, and the power is out	3
Injured	4
Needs dialysis or insulin	12
No drinking water (6 in a heatwave)	12
Nothing above	24
Flood	
Water at chest or roof level	1
Water at waist and rising	2
Water at knee and rising	4
Trapped	6
Heatwave	
Signs of heatstroke: confused, fainted, hot and dry skin	0.5
Signs of heat exhaustion: vomiting, dizzy, cramps	2
Very hot indoors, no cooling, and elderly, infant, ill or alone	6
Very hot indoors, no cooling	12
Earthquake	
Trapped under rubble	1
Smell of gas, or fire	1

Signal in the report	Hours left (demo default)
Inside a badly damaged building	2
Building cracked, people outside	12
Storm or cyclone	
Roof or walls gone with people inside	2
In an unsafe building before landfall	3 (or hours to landfall)
Storm-surge water	use the flood rules
Wildfire	
Fire in sight, or the route out is blocked	1
Heavy smoke with breathing trouble	2
In the evacuation zone without a vehicle	3
Landslide	
Buried or partly buried	1
House cracking or slope moving	3
Road cut, otherwise safe	24

d. Priority points

Points = time points + "can't save themselves" points + window points + people points, then adjusted for trust. Ties go to the older report. A value exactly on a band edge counts in the more urgent band. These rules are the same for every disaster.

Part	Rule	Points
Time left	Under 1 h / 1–3 h / 3–6 h / 6–12 h / over 12 h	50 / 40 / 25 / 10 / 0
Can't save themselves	Cannot walk, or disabled	+15
	Elderly	+10
	Infant, child or pregnant	+10
	Alone	+10
Window closing	Their window closes within 3 hours (see f)	+20
People	+2 per person	up to +10
Trust	Final points = points × (0.6 + 0.4 × confidence)	

Flood example (house 14): oxygen, about 3 hours left: 40. Cannot walk 15, elderly 10, alone 10, one person 2. Subtotal 77. Confidence 0.75 gives 0.6 + 0.4 × 0.75 = 0.9. Final 77 × 0.9 = **69 points**.

Heatwave example (top-floor flat): heatstroke signs, about 30 minutes left: 50. Elderly 10, alone 10, one person 2. Subtotal 72. Confidence 0.8 gives 0.92. Final 72 × 0.92 = **66 points**. Medical needs count through "time left", never twice.

e. Confidence: how much we trust a case (0 to 1)

Source of the first report	Starting confidence
Verified rescue team	0.9
Call-taker who spoke to the person	0.8
App form, helper registered before the disaster	0.7
Free text read by the AI	0.6
App form or alert reply, unknown phone	0.5
Free text read by keyword rules	0.4
"Unconfirmed" case created from silence (no reply to an alert)	0.3

- **+0.15** for each matching report from a *different* phone or source, up to 1.0. Reports from the same phone count once.
- **+0.1** if GPS is accurate to under 50 m; **−0.1** if the location is only a landmark.
- **−0.1** for every 2 hours without an update, down to 0.2. After 3 hours with no update, the case gets a "re-check" badge.
- A single report from an unknown phone claiming more than 10 people starts at 0.3 with a "verify" badge (the fake-report guard). Each phone can send at most 20 reports an hour.
- A dispatcher pressing "Verified" sets confidence to 1.0.

Low confidence **never hides** a case. It moves it down and shows a badge. A case with high urgency but low confidence gets a "verify fast" badge at the top of the list.

f. Window closing: acting before help can't get through

Each profile says what "window closing" means. If a case's window closes within 3 hours, it gets **+20 points** and a card line like "Window closes in about 2 h (Bridge Rd floods)".

Disaster	The window closes when…
Flood	The road to them floods, so trucks can no longer reach them.
Heatwave	The hottest hours start (or a planned power cut begins) and they have no cooling.
Earthquake	Night falls or rain arrives, making rubble search harder.
Storm or cyclone	Landfall arrives and teams must stop moving.
Wildfire	The fire front reaches their only road out.
Landslide	More heavy rain is forecast for their slope.

In the hackathon these times come from our scenario files. In real use they would come from forecasts (for example Open-Meteo's free flood and weather data); that is a stretch goal.

g. Merging duplicates

Two reports are the same case when **all four** are true:

1. They are within 150 m of each other, or give the same landmark.
2. They are within 3 hours of each other.
3. They are about the same disaster type (or one of them doesn't say).
4. They don't contradict each other: people counts within 2, and at least one condition or key word (house number, name) in common, or one of them has no details.

The merged case keeps every report ID, every condition mentioned, the newest answers and the most accurate GPS. It shows a "2 reports" badge. If we get it wrong, the dispatcher splits it with one button.

h. Silent Zones

1. We split the demo town into 8 to 10 named zones (drawn as shapes on a map at geojson.io).
2. The active hazard profile says which zones count as **hard-hit** (inside the flood extent, the heat-alert area, the strong-shaking area, and so on). The scenario gives each zone's population; the registry gives its registered vulnerable people.
3. **Report rate** = the median number of reports per 100 people across the *other* hard-hit zones, so it adjusts itself to any disaster and scenario.
4. **Expected reports** = zone population ÷ 100 × report rate. **Silence ratio** = actual reports ÷ expected reports.
5. A zone is **Silent** when it is hard-hit and either: its silence ratio is below 0.25 (with at least 8 expected reports), or its warning coverage is below 25% (with at least 10 registered vulnerable people).
6. The map paints it red with a card (Figure 7). "Send welfare-check team" creates a zone task worth 60 points, +1 per registered vulnerable person, capped at 90. "Outreach list" sends the unconfirmed names, highest risk first, to volunteers.
7. When replies and reports start arriving from the zone, the numbers rise and the red fades.

Worked example (Ward 4, flood): 2,000 people. The other hard-hit zones produce a median of 1.2 reports per 100 people, so Ward 4 should produce about 24 reports. It produced 2: silence ratio 0.08. Only 9 of 41 registered vulnerable people are confirmed: coverage 22%. **Silent.** Heatwaves are quieter by nature (people rarely report heat), so for heat the warning-coverage test does most of the work.

i. Matching teams and places to cases

1. Each team has a type and a demo speed: boat 10 km/h, truck 25 km/h, medic team 25 km/h, search and rescue 20 km/h, fire crew 25 km/h, cooling and welfare team 20 km/h, volunteer on foot 4 km/h.
2. **Who can help** comes from the profile's needs rules. Examples: waist-deep water needs a boat; trapped under rubble needs search and rescue; heatstroke signs, oxygen, bleeding or injury need a medic; wildfire near or route blocked needs an evacuation vehicle or fire crew; a welfare check can be done by anyone.
3. **Travel time** = straight-line distance × 1.4 ÷ speed. Example: a boat 1.5 km away takes about 13 minutes.
4. **Suggest:** the fastest free team that can help, and the nearest shelter or cooling centre with space if the person needs to move.
5. **Trade-off:** if that team is also the best choice for another case in the top 10, the screen says so: "Boat 2 is also best for case #3. If it goes here, #3 waits about 35 more minutes (next best: Boat 4)."

6. **Shelters and cooling centres:** each has a capacity and a count of people inside, updated by phone or by call. When one is nearly full, the dashboard suggests the next one and offers to update the alert.
7. **Suggest all** walks down the ranked list and gives each case its fastest free team. A person still confirms each one.
8. **Only teams that can really go:** suggestions use teams the tracker marks Available, with fuel not "low" and the supplies the job needs. Teams about to finish show as "free in about 10 min" (section 3.7).

3.10 When things go wrong

What fails	What the system does	What the screen says
The internet	The AI reader switches to keyword rules. Everything else runs on the laptop as normal.	"Offline: keyword rules active" banner
Phone towers	Alerts and reports travel phone to phone; helpers pass warnings on by knocking.	Hop count on each message
No other phone nearby	Messages keep waiting; the QR backup can move them.	"0 phones nearby"
A phone loses its connection	After 2 failed checks it switches to phone-to-phone relay; nothing leaves the outbox until the server accepts it.	"Offline: passing messages phone to phone"
The laptop restarts	The SQLite database file keeps everything.	Nothing lost
Someone sends a fake alert	Phones ignore any alert without the command post's signature.	Never shown
A report is fake	Low starting confidence, a "verify" badge, a per-phone limit; it never jumps the queue unchecked.	"Verify" badge
A report is wrong	The dispatcher overrides it with a reason; the log keeps both versions.	"Changed by dispatcher: <reason>"
Two reports are wrongly merged	The dispatcher splits them with one button.	Two cases again
A team doesn't arrive	Overdue flag at ETA + 50%; "not heard from" after 15 minutes without a ping; the dispatcher calls, reassigns or sends another team.	"Boat 3 overdue" warning
A team's position is old	The icon shows the time of its last position and turns grey after 10 minutes.	"As of 15:41"
A supply runs low	Low-stock warning at the item's level; the burn rate predicts when it runs out.	"Oxygen cylinders: 3 left"
People ignore alerts	Alerts are rare, targeted to zones, and graded by severity; unanswered ones turn into welfare checks.	Coverage bar per zone

3.11 Privacy, safety and fairness

- **Sealed reports:** health details and exact locations are sealed so only the command post can read them. Relaying phones carry them without being able to open them. Team location pings are sealed the same way.

- **Signed messages:** every report is signed by the phone that made it, and every alert by the command post, so nobody can fake or change them on the way. (Lesson from Bridgefy, a mesh app where researchers showed messages could be faked.)
- **Opt-in registry:** only people who agree are on it, with the fewest fields possible. Outreach lists show volunteers only what they need to knock on the right door. The data is deleted after the crisis.
- **Humans decide:** the system suggests; a dispatcher confirms every job and every alert. Overrides need a reason and are logged.
- **Visible, tunable rules:** no hidden scoring. Local authorities can change every weight and every profile.
- **Accessible warnings:** read aloud, local languages, big icons, plain words, and human channels for people who can't use a phone.
- **Fairness check (stretch):** a panel showing the share of urgent cases served and the warning coverage in each area, to catch any area being neglected.

PART 4

How we build it

4.1 Assumptions and golden rules

Assumptions (if any of these is wrong, we adjust the plan together):

- 4 people and about 36 hours of build time. With only 24 hours, we follow the cut rules in 4.8.
- 3 Android phones with Google Play services (plus 1 spare), 1 laptop, and our own Wi-Fi router or a spare phone hotspot. The router does not need internet.
- We build one engine for every disaster type, and fully test two hazard profiles: flood and heatwave.
- Everyone codes with an AI coding tool. Ready-to-paste prompts are in 4.10.

Golden rules

1. **Build the riskiest part first.** Two phones talk to each other in hour 1.
2. **Agree the data formats before anyone codes,** including the alert packet and the hazard profile format. Then all 4 people can work in parallel.
3. **Never hard-code a disaster.** If code says "water" or "flood", it belongs in a profile file instead.
4. **Always have something that runs.** Commit every working step. The main branch always works.
5. **Fake data first, real smarts later.** The dashboard is built on a fake feed from hour 2.

4.2 Who does what

Person	Owns	Pairs with
A: Mobile	Field app: alert screen and read-aloud, report form driven by profiles, phone-to-phone relay, jobs, location pings, outcome form, QR backup	B, on direct sync and signatures
B: Server	API, database, profile engine, scoring, merging, alerts and warning coverage, Silent Zones, team tracking, shelters and supplies	A, on unsealing and signing
C: Dashboard	Map, ranked list, explanation cards, alert composer, intake form, paste box, teams and resources panel, chaos panel	D, on the demo flow
D: Data, AI and pitch	Hazard profiles (flood, heat, earthquake), AI and keyword readers, scenario packs with teams and supplies, announcement templates, simulation, slides	Everyone, for end-to-end testing

With 3 people, C also takes the scenario packs and everyone shares the pitch. With 2 people, cut every Stretch item and most Should items.

4.3 Tools

Part	Tool	Why it is the simplest choice
Field app	Flutter (Dart), Android only	One codebase for screens; AI tools write it well; has a ready Nearby package.
Phone to phone	`nearby_connections` Flutter package (Google Nearby Connections)	Finds nearby phones and sends data over Bluetooth and Wi-Fi with no internet.
Connection changes	`connectivity_plus`	Notices Wi-Fi or mobile data coming and going, so the phone switches between direct and mesh at once.
Public route	`cloudflared` quick tunnel	When the laptop has internet, phones on any Wi-Fi or mobile data can reach the server directly.
Read aloud	`flutter_tts` (the phone's text-to-speech)	Speaks alerts in the phone's installed languages, offline.
Storage on the phone	`sqflite` (SQLite)	Messages survive the app closing.
Location	`geolocator`	GPS works without any signal.
QR backup	`qr_flutter` to show, `mobile_scanner` to scan	Two small packages, no setup.
Signing and sealing	`pinenacl` on the phone, `PyNaCl` on the server	The same libsodium signatures and "sealed box" on both sides.
Server	Python, FastAPI, SQLite	A few files, runs on any laptop, AI tools know it well.
Dashboard	React, Vite, react-leaflet	Quick to start; a map in a few lines.
Offline map	Protomaps `.pmtiles` file of our town + `protomaps-leaflet`	The map loads with no internet. Backup: open the map once while online so tiles are cached.
AI message reader	Claude API (`claude-haiku-5-5`) with JSON output	Fast for short messages; also helps translate alert wording when online.
Live updates	Dashboard asks the server for changes every 2 seconds	Simpler than websockets, and fast enough.
Simulation chart	Python script + matplotlib	Runs once and saves an image for the slide.

Why Android only? Google's Nearby Connections works on Android. Apple has its own system (MultipeerConnectivity) that only connects iPhones to iPhones, and bridging the two is a project of its own. Android is the most common phone system in the world, so it is the right place to start.

4.4 Repo layout

One GitHub repository. Each person owns one folder, so we rarely edit the same files.

```
pulsetriage/
  shared/              formats everyone reads first
    packet.json        the packet, report, alert and case formats
    profiles/          one file per disaster: flood.json, heat.json, earthquake.json, ...
  field/               Flutter app (A)
  server/              FastAPI + SQLite (B)
  dashboard/           React + Vite (C)
  scenario/            scenario packs, announcement templates, simulation (D)
    flood/  heat/  earthquake/
```

4.5 Data formats

We agree these in hour 0, save them in `shared/`, and nobody changes them without telling the whole team. (The `//` comments are explanations; leave them out of the real files.)

1. The packet: what phones pass to each other

```
{
  "id": "8f1c2a7e-...",            // unique ID made on the phone (UUID)
  "kind": "report",                // see the list of kinds below
  "from_device": "dev_4KQ2",       // short ID of the phone (or "command_post")
  "pubkey": "base64...",           // the sender's public key, to check the signature
  "created_at": "2026-10-18T14:05:00Z",
  "expires_at": "2026-10-19T14:05:00Z",  // phones drop it after this
  "hops": 2,                        // +1 each time it is passed on
  "max_hops": 12,                   // stop passing it on after this
  "area": "tsm2k",                  // rough area only: 5-letter geohash, about a 5 km box
  "urgency_hint": 3,                // 4 = alert (always first), then 3 to 0
  "sealed": "base64...",            // private details, sealed to the command post's key
  "public": null,                   // used instead of "sealed" for alerts (readable by all)
  "sig": "base64..."                // signature over every field above
}
```

2. Inside the seal: a report

```
{
  "hazard": "flood",               // flood | heat | earthquake | storm | fire | landslide | other
  "for_whom": "neighbour",         // self | neighbour | other
  "people": 1,
  "conditions": ["elderly", "cannot_walk", "oxygen", "alone"],
  // "details" = answers to the questions in this disaster's profile
  "details": {"water": "waist", "rising": true, "trapped": false},
  "note": "Old man alone, 2nd floor, oxygen tank half empty",
  "lat": 0.0, "lng": 0.0, "gps_accuracy_m": 15,
  "landmark": "behind the green mosque, house 14",
  "reporter_role": "helper",       // helper | volunteer | rescue_team
  "contact": ""                     // optional phone number
}
```

Kinds of packet: `report`, `alert`, `alert_reply`, `welfare_check`, `volunteer`, `job`, `ack`, `team_ping`.

A heatwave report has the same shape with different details, for example `"hazard": "heat", "details": {"hot_inside": true, "cooling": false, "signs": ["confused", "hot_dry_skin"], "power_out": false}`.

Conditions about the person (shared by every disaster; the form shows these as buttons): `elderly`, `child`, `infant`, `pregnant`, `cannot_walk`, `disabled`, `oxygen`, `dialysis`, `insulin`, `power_dependent` (a medical device that needs electricity), `other_medicine`, `injured`, `bleeding`, `unconscious`, `alone`, `no_food_water`.

3. An alert (public, signed by the command post)

```
{
  "alert_id": "A-0007",
  "hazard": "heat",
  "severity": "warning",           // watch | warning | emergency
  "zones": ["Ward 4", "Old Market"],
  "headline": {"en": "Extreme heat today, 12:00-17:00", "local": "..."},
  "do_now": {"en": ["Stay in the coolest room. Drink water often.",
                    "Check on elderly, ill or lonely neighbours twice today.",
                    "Someone confused or fainting? Report it now."], "local": ["..."]},
  "places": [{"name": "Community hall", "type": "cooling_centre",
              "lat": 0.0, "lng": 0.0, "space_left": 120}],
  "issued_at": "2026-10-18T07:00:00Z",
  "valid_until": "2026-10-18T20:00:00Z",
  "replaces": null                  // the alert ID this one updates or cancels (all-clear)
}
```

A reply is a sealed `alert_reply` packet: `{"alert_id": "A-0007", "status": "safe" | "need_help" | "helping", "registry_id": optional}`.

4. A hazard profile (shortened heatwave example)

```
{
  "hazard": "heat",
  "label": {"en": "Heatwave"},
  "questions": [
    {"id": "hot_inside", "ask": "Very hot indoors?", "type": "yesno"},
    {"id": "cooling", "ask": "Fan, AC or a cool place?", "type": "yesno"},
    {"id": "signs", "ask": "Signs of heat illness", "type": "multi",
     "options": ["confused", "fainted", "hot_dry_skin", "vomiting", "dizzy", "cramps"]},
    {"id": "power_out", "ask": "Power cut?", "type": "yesno"}
  ],
  "hours_rules": [
    {"all": ["signs:confused|fainted|hot_dry_skin"], "hours": 0.5, "text": "Signs of heatstroke"},
    {"all": ["signs:vomiting|dizzy|cramps"], "hours": 2, "text": "Signs of heat exhaustion"},
    {"all": ["hot_inside", "!cooling", "cond:elderly|infant|disabled|alone"], "hours": 6,
     "text": "Very hot indoors, no cooling, high risk"},
    {"all": ["hot_inside", "!cooling"], "hours": 12, "text": "Very hot indoors, no cooling"}
  ],
  "needs_rules": [
    {"all": ["signs:confused|fainted|hot_dry_skin|vomiting|dizzy|cramps"], "need": "medic"},
    {"all": [], "need": "welfare"}
  ],
  "window": {"text": "Hottest hours start", "only_if": ["!cooling"]},
  "affected": "alert_zones",
  "keywords": {"signs:confused": ["confused", "not making sense"],
               "hot_inside": ["very hot", "no fan"]},
  "tips": ["..."], "first_aid": ["..."], "alert_templates": {"warning": {"en": "..."}}
}
```

How to read a rule: `hot_inside` means that answer is yes; `!cooling` means it is no; `signs:a|b` means any of those options was ticked; `cond:a|b` means the person has any of those conditions; `water:waist|chest` means the chosen answer is one of those. The common "any disaster" rules live in `profiles/common.json`.

5. The case: what the server keeps after merging

Field	Example	Meaning
`id`	`case_0142`	Case number
`hazard`	flood	Which profile was used
`report_ids`	`[r1, r7]`	Every report merged into this case
`source`	app, call, paste, alert_reply, silence	How the first report arrived
`lat`, `lng`, `accuracy_m`	GPS, 15 m	Best location among its reports
`zone`	Ward 4	Named area, for Silent Zones and coverage
`people`	1	Largest count any report gave
`conditions`	elderly, cannot_walk, oxygen, alone	Every condition any report mentioned
`details`	`{"water": "waist", "rising": true}`	Newest answers to the profile's questions
`hours_to_critical`	3	Time left before it becomes life-threatening
`window_closes_at`	17:00	When help gets much harder, if known
`points`, `reasons`	69, `[{"text": "Cannot walk", "points": 15}, ...]`	Priority points and the explanation card
`confidence`	0.75	0 to 1: how much we trust it
`needs`	boat, medic	Which kinds of team can help
`status`	new, verified, assigned, en_route, done	Where the case stands
`assigned_team`, `shelter`	Boat 2, School	Who is on it and where the person goes
`reader`, `last_update`	form, 15:30	How it was read; for the stale-report check

6. Teams, location pings and supplies

A location ping is a sealed packet of kind `team_ping`. Phones keep only the newest ping per team and drop older ones.

```
{
  "team_id": "boat_2",
  "lat": 0.0, "lng": 0.0, "gps_accuracy_m": 10,
  "at": "2026-10-18T15:41:00Z",      // when the position was taken, not when it arrived
  "status": "en_route",              // available | assigned | en_route | on_scene | returning |
out_of_service
  "case_id": "case_0142",
  "battery": 64,                     // percent
  "fuel": "half"                     // full | half | low (vehicles and boats)
}
```

A job update is an `ack` packet: `{"job_id": "J-031", "status": "en_route" | "arrived" | "done" | "cant_reach", "outcome": {"people_helped": 1, "taken_to": "School shelter", "supplies_used": {"oxygen_cylinder": 1}}, "reason": ""}`.

Team field	Example	Meaning
`id`, `name`, `type`	boat_2, Boat 2, boat	Who it is
`capabilities`	boat, medic	What it can do
`status`, `status_since`	en_route, 15:33	Current state and since when
`case_id`, `eta`	case_0142, 15:46	Current job and expected arrival
`last_ping`	position at 15:41, battery 64, fuel half	Last known position and condition
`load`	`{"oxygen_cylinder": 2, "water_l": 40}`	Supplies on board
`history`	(status, time) pairs	Timings for every step

Supply field	Example	Meaning
`item`, `unit`	drinking water, litres	What it is
`place`	command_post, shelter_school, boat_2	Where it is
`qty`	1200	How much is there now
`warn_at`	300	Low-stock warning level

Every change is a row in `supply_moves`: item, from, to, quantity, time, who, and why (restock, load, delivered, used, transfer).

7. The server API

Call	Who calls it	What it does
`POST /api/packets`	Online phones	Uploads a batch of packets; returns the IDs the server now has.
`GET /api/outbound?since=`	Online phones	Alerts, jobs and replies to push into the mesh.
`GET /api/profiles`	Dashboard, phones	The hazard profiles (phones also get them at setup).

Call	Who calls it	What it does
`POST /api/alerts`, `GET /api/alerts`	Dashboard	Create (sign and queue) or list alerts, including updates and the all-clear.
`GET /api/coverage`	Dashboard	Warning coverage per zone, and the unconfirmed people list.
`POST /api/announcement`	Dashboard	Writes a 20-second loudspeaker or radio script for an alert, per language.
`GET /api/cases`	Dashboard, every 2 s	Ranked cases with reasons.
`GET /api/zones`	Dashboard	Zones with expected vs actual reports, coverage and the Silent flag.
`POST /api/intake`	Dashboard	A report typed by a call-taker.
`POST /api/paste`	Dashboard	A batch of pasted messages for the AI reader.
`GET/POST /api/teams`	Dashboard	List or add teams, with status, job, ETA, last ping and load.
`POST /api/teams/{id}/status`	Dashboard	Dispatcher sets a status by hand (for example after a radio call), with a reason.
`GET /api/supplies`, `POST /api/supplies/move`	Dashboard, phones	Stock per item and place, used today, left, runs out in; restock, load, deliver, use or transfer.
`GET /api/resources/summary`	Dashboard	Counters and warnings for the Teams and resources panel.
`GET/POST /api/shelters`	Dashboard, phones	Shelters and cooling centres, with capacity and how full they are.
`GET /api/suggest?case_id=`	Dashboard	Best team and place for a case, and who would wait longer.
`POST /api/assign`	Dashboard	Assigns a team to a case and creates a job packet.
`POST /api/cases/{id}/override`, `/split`	Dashboard	Dispatcher changes a case (reason required) or undoes a wrong merge.
`POST /api/reports/simulate`	Scenario replay	Adds plain-text reports as if they came in by text or mesh.
`POST /api/chaos/{kind}`	Chaos panel	`duplicate`, `fake`, `wrong_location`, `ai_down`.
`POST /api/registry/import`	Dashboard	Uploads the registry CSV.
`GET /api/status`	Dashboard, phones	Is the server up? Is the AI reader reachable? Packets so far?
`GET /api/setup-qr`	Dashboard	QR code with the server address and the command post's public key.

4.6 Feature list and priorities

Must = the demo fails without it (22). Should = it makes us win (18). Stretch = only after a full rehearsal (7). If we fall behind, we cut from the bottom of each group upwards.

#	Feature	What it does	Priority	Owner	From
1	Hazard profile engine	Questions, time rules, needs and templates come from one file per disaster	Must	B, D	New
2	Flood and heatwave profiles	The two fully tested disaster types	Must	D	New
3	Alerts through the mesh	Dashboard issues; phones show full screen, read aloud; alerts sent first	Must	A, B, C	New
4	Alert replies	"I'm safe / I need help / I'm helping others"	Must	A, B	New
5	Report form from profiles	"What's happening?", then that disaster's questions	Must	A	Concept 2
6	Offline outbox	Messages wait safely on the phone until passed on	Must	A	Concept 2
7	Phone-to-phone relay	Store and forward with no internet, no repeats	Must	A	Concept 2
8	Direct sync when online	With Wi-Fi or mobile data, upload and download straight to the server; mesh only when offline	Must	A, B	Concept 2
9	Server and database	Receives packets and keeps every case	Must	B	Concept 1
10	Call-taker intake form	Phone calls, radio and walk-ins go straight into the system	Must	C	New
11	AI reader + keyword backup	Free text to fields, including which disaster	Must	D	Concept 1
12	Priority points + explanation card	Ranks cases and says why in plain words	Must	B, C	Concept 1
13	Duplicate merging + confidence	Joins repeat reports; more matches means more trust	Must	B	Concept 1
14	Map + ranked list	The dispatcher's main screen	Must	C	Concept 1
15	Silent Zones	Flags hard-hit areas with far fewer reports or replies than expected	Must	B, C	New
16	Team assignment + trade-off	Best free team per case, and who waits longer	Must	B, C	Concept 1
17	Jobs and "help is coming"	Assignments travel out through the mesh; replies come back	Must	A, B	Concept 2
18	Team status board	Available, assigned, en route, on scene, returning, out of service, with times	Must	B, C	New
19	Live team locations	Pings every 2 min on a job; last seen; stale icons go grey	Must	A, B, C	New

#	Feature	What it does	Priority	Owner	From
20	Arrival check + overdue flags	Auto "on scene" within 100 m or "Arrived"; overdue and not-heard-from warnings	Must	A, B	New
21	Supply inventory + low-stock warnings	Stock at command post, shelters and on teams; loaded, used, left	Must	B, C, D	New
22	Scenario packs + replay	Flood and heatwave stories streamed in over about 2 minutes	Must	D	Demo
23	Warning coverage	Who has and hasn't answered; unconfirmed people become welfare checks	Should	B, C	New
24	Vulnerability registry	CSV of opted-in at-risk people; feeds coverage and Silent Zones	Should	B	New
25	Offline banner	Shows when the AI reader is down and rules took over	Should	C	Concept 1
26	Shelters and cooling centres	Capacity, head count, nearest with space; shown in alerts	Should	B, C	New
27	Window-closing points	Cases move up before help can't get through	Should	B, D	Concept 3
28	Announcement script	20-second loudspeaker and radio text per language	Should	D	New
29	Welfare checks + outreach lists	"Checked on house 14: safe"; who to visit first	Should	A	Concept 2
30	Paste box	Forwarded messages read into cases	Should	C, D	New
31	Job outcome form	People helped, where they went, supplies used, or "can't reach"	Should	A	New
32	Burn-rate forecast	"Drinking water runs out in about 9 h at the current rate"	Should	B	New
33	Fuel and battery levels	Teams report fuel; low fuel means no new jobs until refuelled	Should	A, B	New
34	Signed alerts and reports, sealed reports	No fake alerts; relaying phones can't read or fake reports	Should	A, B	Concept 2
35	Chaos button	Injects duplicates, a fake, a wrong location, an AI outage	Should	C, D	Demo
36	Volunteer resources	"I have a boat" or "I am a nurse" become assignable	Should	A, B	Concept 2
37	Earthquake profile	A third disaster type, to prove the engine is general	Should	D	New
38	QR backup	Pass messages by scanning a code if Bluetooth fails	Should	A	Concept 2
39	Stale-report re-check	Old reports lose trust and get flagged	Should	B	Concept 1
40	Comparison simulation chart	First come first served vs PulseTriage, one number for the pitch	Should	D	Demo

#	Feature	What it does	Priority	Owner	From
41	Storm, wildfire, landslide profiles	Settings files only, lightly tested	Stretch	D	New
42	Text messages in and out	An Android phone sends alerts to the registry and forwards incoming texts	Stretch	B	New
43	Automated check-in calls	"Press 1 if safe" calls to the registry	Stretch	D	New
44	Live hazard data	Open-Meteo flood and weather data instead of scripted times	Stretch	D	Concept 3
45	Fairness panel	Urgent cases served and coverage per area	Stretch	C	New
46	Road-aware travel times	OpenStreetMap roads instead of straight-line distance	Stretch	B	Concept 3
47	Offline safety tips library	Every profile's tips browsable in the app	Stretch	A, D	New

4.7 Build roadmap

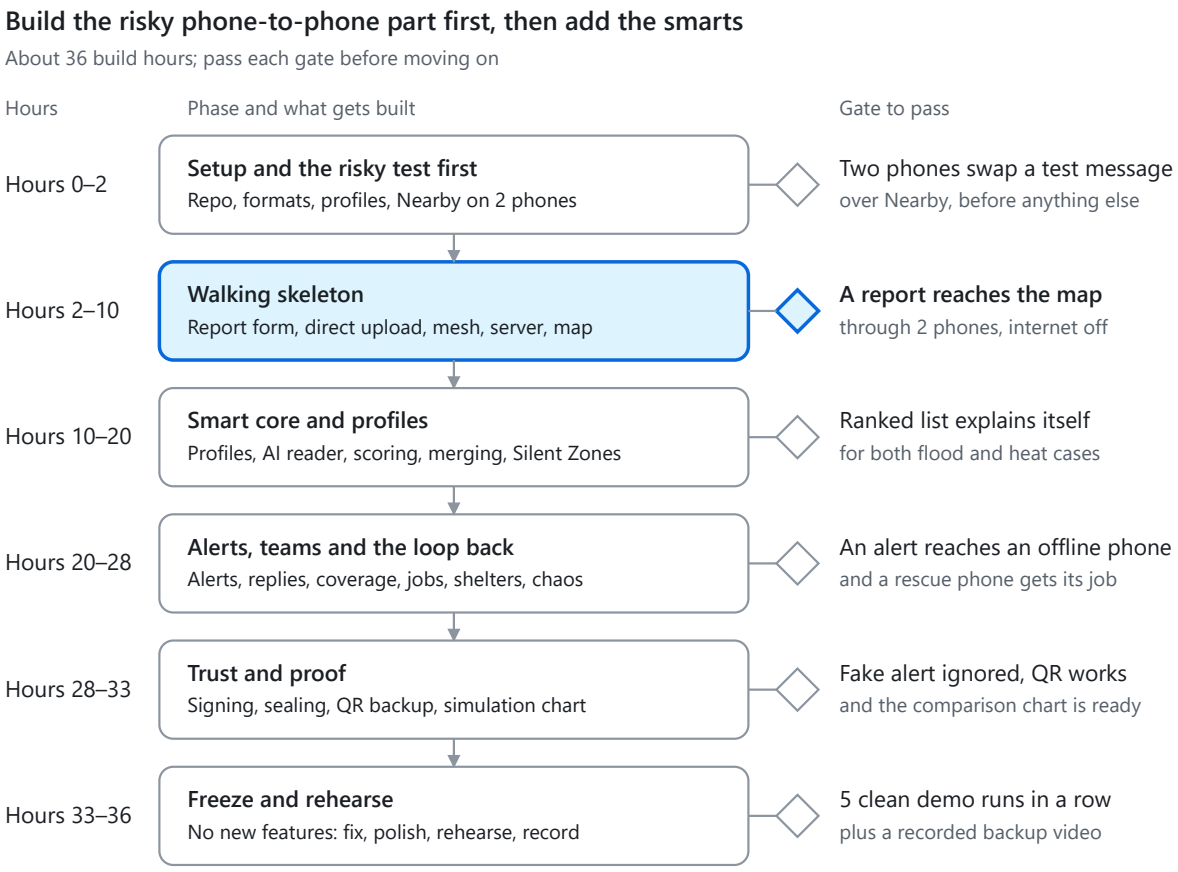


Figure 12. Six phases, each with a gate. The walking skeleton is the gate that matters most: once one report crosses from phone to map with the internet off, everything after it is adding smarts.

4.8 Hour-by-hour tasks

Tick tasks off as you go. Each phase ends with a **gate**: a test that must pass before we move on. If a gate slips by more than 2 hours, we use that phase's cut rule instead of pushing on.

Hours 0–2: setup and the risky test first

- [ ] **All** Create the repo with the folders in 4.4; everyone clones it.
- [ ] **All** Write `shared/packet.json` (packet, report, alert, case) and the profile format from section 4.5 (20 minutes, together).
- [ ] **D** Pick the demo town and draw 8 to 10 zones at geojson.io.
- [ ] **A** Install Flutter and Android Studio; turn on developer mode and USB debugging on all 3 phones.
- [ ] **A** Nearby "hello world": phone 1 sends "hello", phone 2 shows it. **This is the gate.**
- [ ] **B** FastAPI skeleton with SQLite; `GET /api/status` answers; it runs on the laptop at `0.0.0.0:8000`.
- [ ] **B** Set up the router or hotspot, note the laptop's IP address, allow port 8000 through the firewall.
- [ ] **C** Vite + React + react-leaflet app showing a map centred on the demo town.
- [ ] **D** Get a Claude API key and test one extraction call from a small script.

Cut rule: if hello world isn't working by hour 3, A spends at most one more hour, then switches to QR-only relay and keeps Bluetooth "in progress".

Hours 2–10: walking skeleton

- [ ] **A** Report form screen that builds its questions from a profile file (flood first); saves to the `sqflite` outbox.
- [ ] **A** Mesh loop steps 1 to 7 (have list, want list, send, store).
- [ ] **A** Connection modes: upload directly when the server answers; mesh only when it doesn't.
- [ ] **B** `POST /api/packets` stores packets, opens `sealed` (plain base64 for now), makes one case per report.
- [ ] **B** `GET /api/cases` returns cases (fake points for now); `GET /api/profiles` serves the profile files.
- [ ] **B** `POST /api/reports/simulate` and `POST /api/intake`.
- [ ] **C** Cases as map pins coloured by points; ranked list beside the map; refresh every 2 seconds.
- [ ] **C** Case detail panel with an empty explanation card; call-taker intake form.
- [ ] **D** `flood.json` and `common.json` profiles, version 1.
- [ ] **D** Flood scenario v1: 100 people, 3 hard-hit zones, 50 messages saved as JSON; replay script at 10× speed.
- [ ] **D** Extraction prompt v1 working on 20 sample messages.

Gate at hour 10: phone A makes a report with the internet off, it hops through phone B, phone C on the router uploads it, and a pin appears on the map. **Cut rule:** if it still fails at hour 12, freeze the mesh at whatever works (even 2 phones) and move everyone on.

Hours 10–20: the smart core and profiles

- [ ] **B** Profile engine: read hours rules and needs rules from the profile files (section 4.5, part 4).
- [ ] **B** Priority points and reasons; confidence rules; duplicate merging.
- [ ] **B** Silent Zones maths and `GET /api/zones`; registry CSV import.
- [ ] **B** 10 hand-written test cases (5 flood, 5 heat) with the points you expect.
- [ ] **C** Explanation card: one line per reason with points, the trust line, badges.
- [ ] **C** Zone layer with Silent Zones in red; hazard selector in the top bar; offline banner.

- [ ] **D** (with B) Plug the AI reader into the server, with hazard detection, the keyword backup and the 4-second timeout.
- [ ] **D** `heat.json` profile; heatwave scenario pack (registry-heavy, few reports).
- [ ] **D** Flood scenario v2: full town, 300 to 500 messages, duplicates, 1 fake, 1 wrong location, 1 Silent Zone.
- [ ] **D** Registry CSV for the demo town (made-up names); safety tips and alert templates for flood and heat.
- [ ] **A** Alert screen: full screen, sound, read aloud with `flutter_tts`, three reply buttons, "pass it on".
- [ ] **A** Alert packets get urgency 4 and are always sent first.
- [ ] **A** Mesh reliability (retries, reconnects); test with 3 phones in different rooms.

Gate at hour 20: run the flood replay: the list ranks with readable cards, duplicates merge, a Silent Zone lights up. Switch to the heat pack: heat cases rank with heat reasons. **Cut rule:** drop the weights panel and the registry import (put the registry straight into the scenario files).

Hours 20–28: alerts, teams and the loop back

- [ ] **B** `/api/alerts`: create, queue to outbound; alert replies update **warning coverage**; unconfirmed people become cases after 30 minutes or 2 hours.
- [ ] **B** Teams, `/api/suggest` with the trade-off message, `/api/assign` creating job packets.
- [ ] **B** Team tracker: status changes with times, `team_ping` ingest, arrival within 100 m, overdue and not-heard-from checks every minute.
- [ ] **B** Supplies table and `supply_moves` log; `/api/supplies`, `/api/supplies/move`, `/api/resources/summary`; low-stock warnings.
- [ ] **B** Shelters and cooling centres; window-closing points; stale-report re-check; chaos endpoints.
- [ ] **C** Alert composer: template, severity, zones, steps, places, languages, phone preview.
- [ ] **C** Teams and resources panel: counters, team table, supplies table, warnings bar.
- [ ] **C** Team icons on the map coloured by status, with last-seen time, a line to their case and a short trail; grey when stale.
- [ ] **C** Coverage per zone on the map; status steps on the card; chaos panel; paste box.
- [ ] **D** Team start positions and a supplies list with starting stock in both scenario packs; simulated pings that move scenario teams during replay.
- [ ] **D** Window times in both scenario packs; announcement script templates (AI helps translate when online).
- [ ] **D** Start the comparison simulation script; first draft of the slides.
- [ ] **A** Online phones download alerts and jobs from `/api/outbound` and pass them to offline neighbours.
- [ ] **A** Jobs screen (On my way, Arrived, Done, Can't reach) with the outcome form; "Help is coming" on the reporter's phone.
- [ ] **A** Location pings every 2 minutes on a job and 10 minutes idle, with battery and fuel; the newest ping replaces older ones in the mesh.
- [ ] **A** Welfare-check screen and outreach list.

Gate at hour 28: an alert issued on the dashboard reaches an offline phone through 2 hops; tapping "I'm safe" updates that zone's coverage; a job reaches a rescue phone, the team shows "on scene" and comes back "done" with an outcome that changes the supply counts; a scenario team that stops moving gets an overdue flag. The chaos button injects all 4 problems and each is handled.

Hours 28–33: trust and proof

- [ ] **A+B** Signing on both sides; alerts signed by the command post and checked by phones; sealing of reports.
- [ ] **B** Rescue-team keys marked as trusted.
- [ ] **A** QR backup: show and scan.
- [ ] **C** Polish for a projector: big fonts, clear colours, loading states.
- [ ] **D** `earthquake.json` profile and a short earthquake pack, if time allows.
- [ ] **D** Finish the simulation and its chart (section 5.4); finish the slides and the demo script.

Gate at hour 33: an unsigned test alert is ignored by every phone, a QR scan moves a message, and the chart is in the slides.

Hours 33–36: freeze and rehearse

- [ ] **All** Feature freeze: only bug fixes from now on.
- [ ] **B** Write `reset_demo.py`, which puts the database back to the demo starting state for a chosen pack.
- [ ] **All** Tag the working version: `git tag demo-v1`.
- [ ] **All** Rehearse the full demo 5 times and time each run.
- [ ] **All** Record a backup video of one clean run.
- [ ] **All** Charge everything and print the setup QR.

> **If we only have 24 hours:** merge the 20–28 and 28–33 phases; keep signed alerts but skip sealing; skip the earthquake profile, shelters, the paste box, the announcement script, window points and multi-code QR; show them on a "next steps" slide. Never cut: alerts, replies, flood and heat profiles, the intake form, Silent Zones, team status, live team locations, the arrival check and supply counts.

4.9 How we work as a team

- **Stand-up every 4 hours, 10 minutes:** what works, what's blocked, what gets cut. The owner of the part decides.
- **The 45-minute rule:** anyone stuck for more than 45 minutes asks the team or switches tasks.
- **Git:** each person works in their own folder. Commit small and often with clear messages. Never push code that doesn't run to the main branch. Tag every version that demos well.
- **Shared formats are sacred:** changing `shared/` needs a message to the whole team first. New disaster types are new profile files, never new code.
- **Test end to end often:** every gate is tested by someone who didn't build that part.
- **Sleep in shifts:** at least 3 to 4 hours each. A and B never sleep at the same time during the walking skeleton (hours 2 to 10).
- **Keep a decisions note:** one shared file listing every decision and cut, with the reason. It becomes our trade-offs slide.

4.10 Prompts for your AI coding tool

Paste the context prompt first in every new session, then one task prompt at a time. Ask for small steps, run the code after every change, and paste the matching section of this brief where a prompt says `<paste ...>`.

Context (everyone, every new session)

```
We are building PulseTriage for a hackathon: a disaster warning and triage system for
any disaster type (flood, heatwave, earthquake, storm, wildfire, landslide). An Android
Flutter app shows alerts and uploads reports straight to the server when it can
reach it, relaying them phone to phone only when it can't; a
command-post laptop runs a FastAPI + SQLite server and a React dashboard that issues
alerts and ranks cases with visible points. Everything disaster-specific lives in
hazard profile JSON files in shared/profiles/; never hard-code a disaster in code.
Read shared/packet.json and the profile files first and never change their fields.
Keep code simple: few files, no extra frameworks, clear names.
```

A1. Nearby hello world (hour 0)

```
Create a minimal Flutter Android app using the nearby_connections package.
On launch, request Bluetooth scan/advertise/connect, nearby Wi-Fi devices and
fine location permissions, and add them to AndroidManifest.xml. Start advertising
and discovery at the same time with Strategy.P2P_CLUSTER and service ID
"pulsetriage". Auto-accept connections. When connected, send "hello from <device
model>" as a bytes payload and show received messages in a list. Show a log of
every discovery and connection event on screen.
```

A2. Report form from profiles, and outbox

```
Add a report form. First question: "What's happening?" with one chip per profile in
assets/profiles/*.json (default: the hazard of the latest alert). Then: for whom,
a -/+ people counter, condition buttons from shared/packet.json, then the selected
profile's "questions" (yesno = switch, choice = chips, multi = multi-select chips),
an optional note and landmark, and GPS from geolocator. On Send, build a packet:
uuid id, kind "report", expires_at +24h, hops 0, max_hops 12, area = 5-character
geohash, urgency_hint 0-3 from the profile's hours rules (under 1 h = 3, under 3 h
= 2, any condition = 1, else 0), sealed = base64 of the report JSON. Save it with
sqflite and show the outbox count on the home screen.
```

A3. Connection modes and mesh sync

```
Replace the hello exchange with this protocol. On connect, send
{"type":"have","ids":[...]}. On "have", reply {"type":"want","ids":[missing]}.
On "want", send {"type":"packets","items":[...]} in batches under 30 KB, sorted by
urgency_hint high to low (alerts are 4), then newest first. On receiving packets,
drop expired ones, ones with hops >= max_hops and ones already stored; save the
rest with hops + 1. When both sides finish, disconnect, wait 15 s and discover again.
Add connection_service: every 10 s offline / 30 s online, and on every
connectivity_plus change, GET {server}/api/status (2 s timeout) on each server URL.
Online after 1 good check: stop discovery, advertise as "PT1|on|<id>", POST all
undelivered packets to /api/packets, mark returned IDs delivered, then poll
GET /api/outbound?since=<last> every 10 s. Offline after 2 failed checks:
advertise "PT1|off|<id>", discover, and connect to "on" phones first.
```

A4. Alert screen

```
When a packet of kind "alert" arrives and its signature matches the command post
public key saved at setup, show a full-screen alert: a colour band per severity
(watch, warning, emergency), the headline and up to 3 "do_now" steps in the chosen
language, the nearest place from "places", a "Read aloud" button using flutter_tts,
and three big buttons: "I'm safe", "I need help" (opens the report form with the
alert's hazard), "I'm helping others". Each tap creates an "alert_reply" packet.
Play a sound and vibrate. Ignore alerts without a valid signature. Add a "Pass it
on" line: "Check on neighbours who are elderly, ill, alone or have small children".
```

A5. Team phone: pings, arrival and outcome

```
For the rescue_team and volunteer roles: every 2 minutes while a job is open (10
minutes when idle), make a sealed "team_ping" packet with team_id, lat, lng,
accuracy, "at" (time the position was taken), status, case_id, battery percent and
fuel (full/half/low, picked by the user). In the outbox and when receiving from the
mesh, keep only the newest team_ping per team_id. On the Jobs screen add buttons:
On my way, Arrived, Done, Can't reach. Set "arrived" automatically when the phone is
within 100 m of the case. Done opens an outcome form: people helped, taken to
(shelter list cached at setup, hospital, stayed), supplies used (item + count). Each
button sends an "ack" packet. Add Out of service (with a reason) and Ready buttons.
```

B1. Server skeleton

```
Create a FastAPI app with SQLite in at most 6 files. Tables: packets, reports,
cases, alerts, replies, teams, shelters, zones, registry, log. Implement these
endpoints: <paste the server API table from section 4.5>. For now, /api/packets
decodes "sealed" as base64 JSON, stores the report and creates one case per report.
Run with uvicorn on 0.0.0.0:8000 with CORS open.
```

B2. Profile engine and scoring

```
Write profiles.py to load shared/profiles/*.json and evaluate rules: a rule's "all"
list matches when every token matches; "x" = details[x] is true, "!x" = false,
"x:a|b" = details[x] equals or contains any of a, b, "cond:a|b" = conditions contain
any of a, b. Write scoring.py: hours_to_critical(case) = smallest "hours" among
matching rules in common.json and the case's hazard profile; points(case) ->
(total, reasons) and confidence(case, reports) using exactly these rules: <paste
section 3.9 d and e>. Write pytest tests for 5 flood and 5 heat cases: <paste>.
```

B3. Merging and Silent Zones

```
Write merge.py and zones.py using these rules: <paste section 3.9 g and h>.
merge.py attaches a new report to an existing case or creates a case, and
recomputes points. zones.py computes expected reports, the silence ratio, warning
coverage and the silent flag for every zone, using the active profile's "affected"
setting. Call it after every new report or reply. Add pytest tests.
```

B4. Alerts and warning coverage

```
POST /api/alerts takes the alert JSON from section 4.5, signs it with the command
post's Ed25519 key (PyNaCl), wraps it in a packet with urgency_hint 4 and adds it to
the outbound queue. Each alert_reply links to an alert and, if possible, a registry
person (by registry_id or by matching location and name). GET /api/coverage returns,
per zone, registered people, confirmed people and the list of unconfirmed people,
highest risk first. Every minute, unconfirmed people become "unconfirmed" cases at
confidence 0.3 after 30 minutes (emergency) or 2 hours (warning).
```

B5. Team tracker and supplies

```
Add teams.py and supplies.py. Teams have status (available, assigned, en_route,
on_scene, returning, out_of_service) with a history of (status, time). On each
team_ping: store it as last_ping (use its "at" time, not the receive time), keep the
last 5 positions, and if status is en_route and the position is within 100 m of the
case, set on_scene. Every minute: flag overdue when now > eta + max(50% of travel
time, 10 min), and "not heard from" when an en_route team has no ping for 15 min.
On ack "done", apply the outcome: case done, shelter head count up, supplies used
moved out of the team's load. Supplies: table (item, unit, place, qty, warn_at) and
a supply_moves log; endpoints GET /api/supplies, POST /api/supplies/move,
GET /api/resources/summary (counters per status and type, warnings, and runs-out-in
from the last 3 hours of use). /api/suggest only offers available teams with fuel
not "low" and the supplies the job needs.
```

C1. Dashboard

```
Create a React + Vite app with react-leaflet and plain CSS. Top bar: hazard
selector, offline banner from /api/status, buttons New alert, Intake, Paste, Replay,
Chaos. Left 60%: map with case pins coloured by points (red 70+, orange 40-69,
yellow under 40), zone polygons from /api/zones coloured by coverage and filled red
when silent, shelters and teams. Right 40%: ranked list from /api/cases refreshed
every 2 s. Clicking a case opens an explanation card with each reason and its
points, the trust line, badges, and buttons Verified, Override (asks for a reason),
Split and Suggest team. Big fonts for a projector.
```

C2. Alert composer, intake form and paste box

```
Add three dialogs. New alert: hazard, severity, zone multi-select from /api/zones,
headline and up to 3 steps pre-filled from the profile's alert_templates, places
from /api/shelters, languages, valid until, a phone-screen preview, and "Generate
announcement script" (POST /api/announcement). Intake: the report form fields for
the chosen hazard plus free text and caller details, posting to /api/intake. Paste:
a big text box posting to /api/paste and listing the cases created.
```

C3. Teams and resources panel

```
Add a "Teams and resources" tab. Top: counters from /api/resources/summary (teams by
status; "Boats 1 of 3 free"; red when a type has none free) and a warnings bar
(overdue, not heard from, low stock). Team table: name, type, status, job, ETA,
last seen ("as of 15:41"), battery, fuel; buttons to set status with a reason.
Supplies table: item, command post, shelters, on teams, used today, left, runs out
in, low-stock flag; a form to restock, load, deliver or transfer. On the map, show
teams as icons coloured by status with a line to their case and their last 5
positions; grey the icon when its position is over 10 minutes old.
```

D1. AI message reader

```
Write extract.py with extract(text, active_hazard) -> dict matching the inner report
schema plus "confidence" (0 to 1) and "reader" ("ai" or "keywords"). Call the
Claude API with model claude-haiku-5-5 and a system prompt that says: decide which
hazard the message is about (flood, heat, earthquake, storm, fire, landslide, other);
output only JSON with the shared fields and that profile's "details" fields; never
invent details; unknown means null. Time out after 4 s. On any error or timeout,
use keyword_extract(text) with the "keywords" from every profile plus the shared
word lists; if unsure of the hazard, use active_hazard.
```

D2. Scenario packs

```
Write scenario/generate.py with a fixed random seed and a --pack option (flood,
heat, earthquake). Read zones.geojson and write per pack: people.json, registry.csv
(200 made-up vulnerable people with conditions), teams.json (with start positions and loads),
shelters.json, supplies.json,
windows.json (times when each zone's window closes) and messages.json. Follow this
story: <paste the pack's story table from section 5.2>. About 20% of messages get
a duplicate within 150 m and 1 hour. Each message has a hidden truth
{severity, deadline_minutes}. The heat pack has few messages and relies on the
registry and alert replies.
```

D3. Simulation chart

```
Write scenario/simulate.py. For each pack, using messages.json, teams.json and the
server's travel-time rule, run two ways of choosing: first come first served, and
PulseTriage's ranking (import scoring.py). Check whether a team arrived before each
case's hidden deadline. Run 20 seeds and save a matplotlib chart of the share of
critical cases reached in time for each way and each pack, with the range.
```

PART 5

The demo and the pitch

5.1 Demo hardware

Device	Role in the demo	Setup
Phone A	Sam's helper phone	Airplane mode with Bluetooth on and no Wi-Fi network joined, so it is offline. Role: Helper.
Phone B	Volunteer and Boat 2's team phone; relays	Same as A. Role: Rescue team (boat with a medic).
Phone C	Online phone	Joined to our router, so it sends directly and uploads what offline phones hand it. Any role.
Phone D	Spare	Same build installed, charged, ready.
Laptop	Command post	Server, dashboard, database, offline map and both scenario packs; connected to our router; projector cable.
Router or hotspot	Command-post Wi-Fi	No internet needed. Our own, so venue Wi-Fi can't block us.

5.2 Scenario packs

A scenario pack is a made-up disaster in a real-looking town: zones, people, a registry, teams, shelters, window times and a script of messages. All people and messages are invented. We build two packs fully and one if time allows.

- **The town:** a real area we know, so the map looks real. 8 to 10 zones with 800 to 3,000 people each, about 15,000 in total (scenario numbers, not census data).
- **The registry:** 150 to 250 opted-in vulnerable people with made-up names and conditions (oxygen, dialysis, wheelchair, bedridden, lives alone, power-dependent device).
- **Teams:** 3 boats, 2 trucks, 2 medic teams, 1 search-and-rescue team, 2 cooling and welfare teams, plus up to 4 volunteers who join from the app. Each has a start position and a load; the replay moves scenario teams with simulated pings.
- **Places:** 2 shelters and 2 cooling centres with capacities.
- **Supplies:** starting stock and a warning level for drinking water, food packs, first-aid kits, oxygen cylinders, insulin, blankets, life jackets, fuel, and oral rehydration salts (heat pack).

Pack 1: flood (the main demo)

The replay speed is adjustable: at 60×, the 2.5-hour story plays in about 2.5 minutes.

Scenario time	What happens	Feature it shows
14:00	River rising; 2 riverside zones are hard-hit	Map and first reports
14:10	Flood warning issued for Ward 4 and Riverside; replies start	Alerts, replies, coverage

Scenario time	What happens	Feature it shows
14:30	150 messages in 30 minutes, 40 of them duplicates	Merging and ranking
15:00	Phone towers fail in Ward 4: no messages or replies from there	A Silent Zone appears; coverage stays low
15:15	Bridge Rd will flood at 17:00	Window-closing points
15:30	"50 people trapped at the stadium" from an unknown phone	Fake-report guard
15:45	A report with GPS 2 km off but the right landmark	Confidence and location check
15:50	Boat 3 stops moving 1.2 km from its case and stops sending pings	Overdue flag; the dispatcher reassigns
16:00	Internet cut at the command post	Keyword rules and the offline banner
16:10	Live on stage: a report from Ward 4 arrives through the mesh	Mesh relay; the Silent Zone fades
16:20	Boat 2 assigned; its icon moves; "on scene" within 100 m; done with an outcome	Team tracking and the loop back
16:25	Oxygen cylinders drop to 3 (warn at 5)	Low-stock warning; burn rate

Pack 2: heatwave (the 20-second switch)

Scenario time	What happens	Feature it shows
07:00	Heat warning for 4 zones, 12:00–17:00; cooling centres listed	Alerts in two languages, announcement script
07:00–09:00	Replies, calls and welfare checks confirm most registered people	Warning coverage
09:00	Unconfirmed people become welfare-check cases; one zone's coverage stays below 25%	Silence becomes cases; Silent Zone
10:15	A volunteer finds an elderly woman with heatstroke signs	Heat profile scoring; medic suggested
11:30	One cooling centre nearly full	Shelter capacity; alert updated with the next centre

Pack 3: earthquake (if time allows)

Strong shaking at 03:10 across 3 zones; towers down in one; reports of people trapped, gas smells and damaged buildings; search-and-rescue teams are the scarce resource; night falling is the window. It shows the same dashboard with a third set of questions and rules.

5.3 Making the fake data

1. Write about 40 message templates per pack by hand, the way people really text in a panic: short, misspelled, mixed languages, no punctuation. For example:
    - Flood: "pls help water till waist my mother 78 cant walk oxygen cylinder only 2 hr left street 5 near school"

- Flood: "we r 6 ppl on roof baby with us"
- Heat: "my dad is very hot and talking strange, no fan, power gone since morning"
- Heat: "is the cooling centre in old market open?" (not a rescue request)
- Earthquake: "building collapsed on 3rd street people under it we hear voices"
- The same family reported by 3 neighbours with different details (tests merging)

2. Ask the AI for 10 variations of each template, once, while online. Save them to `scenario/<pack>/messages.json`, so the demo never needs internet to create data.
3. The generator gives each message a time and a place inside its zone. About 20% get a duplicate within 150 m and 1 hour. The silent zone gets none after its outage.
4. Each message carries a hidden "true" situation and deadline, used only by the simulation.
5. Run the AI reader over all scenario messages ahead of time and save the results, as a backup if the AI service is slow on the day.

5.4 The comparison simulation

This gives us honest numbers for the pitch: how much better PulseTriage's order is than first-come-first-served, for each disaster type.

1. Same messages, same teams, same travel times for both ways.
2. **Way 1:** teams take cases in the order the messages arrived. (In the heat pack, people who never report are never reached.)
3. **Way 2:** teams take cases in PulseTriage's order, including welfare checks created from silence.
4. For each case, check whether a team reached it before its hidden deadline. Both ways are judged by the hidden truth, not by PulseTriage's own points.
5. Measure the share of critical cases reached in time, and the average wait for the most vulnerable 10%.
6. Run 20 random seeds per pack; report the average and the range.
7. In the pitch, always say "in our simulation". Never present it as a real-world result.

5.5 The live demo, minute by minute

A 7-minute run sheet. Adjust to our actual slot by shortening the replay. One person talks; one person clicks; one person handles the phones.

Time	On screen	What we say (in our own words)
0:00–0:40	Slide: the Zhengzhou flood spreadsheet	In 2021, a student's spreadsheet became a flooded city's rescue system. It worked, until it had thousands of rows, duplicates and no way to tell what was still true.
0:40–1:10	Slide: "The quietest people are in the most danger"	Heat kills alone indoors. People under rubble can't call. Any system that ranks messages ranks the people who could already reach you.
1:10–1:30	Dashboard. Phones on the table in airplane mode.	Everything you're about to see runs with no internet, for any disaster.

Time	On screen	What we say (in our own words)
1:30–2:00	New alert: flood warning for Ward 4. Phones buzz one after another; one reads it aloud; tap "I'm safe"; coverage bar moves.	Warnings travel phone to phone, even without towers. And we know who has answered.
2:00–2:40	Start the flood replay. Map fills, list ranks. Click the top case.	Read the explanation card aloud: every point is visible. The AI only reads; the rules decide.
2:40–3:10	"2 reports" badges. Chaos: fake stadium report, wrong location.	Bad data is normal in a disaster. Duplicates merge; the fake gets a verify badge.
3:10–3:40	Ward 4 turns red: Silent Zone card with 22% coverage.	No messages isn't good news. 41 vulnerable people, and silence. We send someone to knock.
3:40–4:20	Live: Phone A (offline) report → Phone B → Phone C (online) → appears in Ward 4; red fades.	No towers. The report walked here, phone by phone. With Wi-Fi or data it would have come straight in.
4:20–4:50	Suggest team → trade-off → Confirm → Phone B shows the job → Arrived → Done with outcome. Counters and oxygen stock update; Boat 3's overdue flag is visible.	Every choice has a cost, and we show it. A person decides, and we always know where every team is.
4:50–5:20	Switch to the heatwave pack. Coverage map; unconfirmed elderly become welfare checks; a heatstroke case jumps to the top.	Same engine, different disaster. In a heatwave nobody calls, so silence is the main signal.
5:20–5:40	Pull the laptop's internet. Banner appears; new messages still get ranked.	The AI is gone; keyword rules took over. Nothing stopped.
5:40–6:30	Slides: simulation chart, then trade-offs.	In our simulation, more critical cases were reached in time. Here is what we chose not to build, and why.
6:30–7:00	Closing slide	PulseTriage finds the people who can't call for help.

5.6 Lines to use in the pitch

- **Opener:** "In 2021, a student's spreadsheet became a flooded city's rescue system. It worked until it had thousands of rows, duplicates and no way to tell what was still true. PulseTriage is that spreadsheet, built to hold up."
- **The insight:** "The quietest people are in the most danger. So we listen for silence."
- **On heat:** "Heat kills quietly, often alone, indoors. Chicago learned that knocking on doors saves lives. We tell volunteers which doors to knock on first."
- **On warnings:** "Sending a warning isn't the goal. Knowing who heard it is."
- **On every disaster:** "One engine. A small settings file per disaster. Today: flood and heat."
- **On the app:** "France learned that an alert app nobody installs is useless. Ours is for the helpers, not the victims."
- **On trust:** "The AI reads. The rules decide. A human confirms."
- **Close:** "PulseTriage finds the people who can't call for help."

5.7 Our trade-offs

We chose	Instead of	Why
One engine + hazard profiles; flood and heat fully tested	A separate app per disaster, or every disaster half-done	New disasters are settings files; two well-tested ones prove it works.
Complement official warnings	Replace cell broadcast	Governments own broadcast systems; we cover the people, places and moments they miss.
Android only	Android and iPhone	A mixed mesh is a project of its own; Android is the most common phone system worldwide.
Visible rules for ranking	A machine-learning model	Responders can trust and tune it; it works offline; we have no real training data.
AI only reads messages	AI decides priority	Explainable decisions; no invented reasons.
Helpers carry the app	Victims install the app	People don't install apps before disasters (France's SAIP lesson).
Mesh, door knocks, loudspeaker scripts	Mass texts and robocalls	Work when towers are down and cost nothing; texts and calls are stretch goals.
Scripted hazard times	Live forecasts	Reliable demo; live data is a stretch goal.
Straight-line travel times	Real road routing	Good enough to show the trade-off.
Opt-in registry	Full population data	Privacy over coverage; Silent Zones partly make up for the gap.
Human confirms every alert and job	Automatic dispatch and alerts	People stay accountable for life-and-death choices.
Simulated comparison	Real-world results	We have no real dispatch data; we say so plainly.

5.8 Tough questions and our answers

Question	Our answer
Is this only for floods?	No. The engine is the same for every disaster; a hazard profile changes the questions, time rules, teams and alert wording. We demo flood and heatwave live, and we have an earthquake profile.
How do you warn people who don't have the app or a phone?	The app is for helpers; every alert asks them to knock on doors. The dashboard writes loudspeaker and radio scripts and gives volunteers a list of who to visit first. Where official cell broadcast exists, it reaches every phone; we'd connect to it.
How do you know people got the warning?	Replies, welfare checks and calls confirm registered vulnerable people. Coverage per zone shows who hasn't answered, and they become welfare checks automatically.
Couldn't someone send a fake alert and cause panic?	Phones only show alerts signed with the command post's key, saved when the phone was set up. Anything else is ignored.
Won't too many alerts make people ignore them?	Only the command post issues alerts; they target specific zones, have three severity levels, expire, and get a clear all-clear.

Question	Our answer
Won't victims just not install it?	They don't need to. Someone with no phone gets warned by a helper's knock and counted when a neighbour reports them; Silent Zones catch areas nobody reports from.
What stops fake reports?	Signed packets, a per-phone rate limit, low starting trust for unknown phones, a verify badge for big claims, and a person confirming every job.
How do you know a team actually got there?	Team phones send location pings. The dashboard marks a team "on scene" automatically within 100 m of the case, or when they tap Arrived. A late or silent team gets an overdue or not-heard-from flag.
Isn't storing health data dangerous?	The registry is opt-in with minimal fields and is deleted after the crisis. Reports are sealed so only the command post can read them. Smart911 in the US and Japan's evacuation lists already work this way.
Why not let the AI rank cases?	Responders need to see why. Our points are visible, tunable, and still work offline. The AI only turns messy text into fields.
Bluetooth range is short. Does a mesh really work?	Messages move with people (store and forward), so they don't need a continuous chain. It works best in dense neighbourhoods; in sparse areas it is slower, which we admit. QR codes and online phones help.
Where do your weights come from?	They are demo defaults, on screen and adjustable, so local authorities and responders set them.
Your simulation proves nothing real.	Correct, and we say so. It compares two ways of choosing on the same data, judged by a hidden truth. Real validation needs historical dispatch data.

PART 6

Who has tried this before

Real systems around the world already do parts of what PulseTriage does, for different disasters. They give us credibility ("this is a real need") and lessons. Sources are listed at the end.

Where	Example	What it is	Lesson for us
Warning people			
Bangladesh	Cyclone Preparedness Programme	Run by the government and the Bangladesh Red Crescent. Volunteers carry cyclone warnings door to door with megaphones, hand sirens and signal flags. More than 70,000 volunteers were mobilised for Cyclone Amphan in 2020.	People carry warnings where technology doesn't reach. Our "pass it on" prompts and loudspeaker scripts.
India	Ahmedabad Heat Action Plan (2013)	India's first city heat action plan: 7-day heat forecasts, colour-coded alerts, outreach to vulnerable people, drinking water and trained hospital staff. A study estimated about 2,380 deaths avoided in 2014 and 2015.	Heat warnings work when paired with outreach to the vulnerable.
USA	Chicago heatwaves, 1995 and 1999	In 1995, medical examiners linked 485 deaths to the heat (739 excess deaths), many of them old, poor and living alone. In a 1999 heatwave the city warned residents, opened cooling centres and checked on seniors door to door, and far fewer died.	Knocking on the right doors saves lives. Our warning coverage and welfare checks.
France	SAIP alert app → FR-Alert	The government alert app (2016–2018) depended on downloads and took over 3 hours to alert during the 2016 Nice attack. Replaced in 2022 by FR-Alert, which uses cell broadcast and needs no app.	An app victims must install gets ignored. Ours is for helpers; we complement broadcast.
Japan	Earthquake early warnings	The Japan Meteorological Agency's warnings reach TV, radio and mobile phones; carriers have sent them to phones since 2007.	Official broadcast is fast but needs working towers. We cover the gaps.
Russia	EMERCOM app and OKSION	OKSION is a national system of alert screens and speakers in crowded places; the emergencies ministry's app pushes warnings, sent to several channels from one console.	Send one alert to every channel at once.
Finding and helping vulnerable people			
Japan	Evacuation-support lists	Since 2013, towns must keep a list of residents who can't evacuate alone; a 2021 change asks them to try to make individual plans. A 2021 study found only about 10% had such plans.	A registry is normal, even required by law. The gap is turning it into live priorities.

Where	Example	What it is	Lesson for us
France	Heatwave registers	Town halls keep a voluntary list of isolated elderly and disabled residents and phone them during heatwaves. (Not re-checked for this brief.)	The model for our registry outreach and check-in calls.
USA	Smart911	People create a safety profile (conditions, disabilities, mobility). A 911 dispatcher sees it only when that person calls; it stops being shared if not refreshed for 6 months.	Privacy design: opt-in, shown only in an emergency, stale data expires.
Russia	LizaAlert	Volunteer search and rescue founded in 2010; gives priority to children, elderly people and people lost in the wild; runs a 24/7 hotline.	Vulnerability-first rules work at scale with volunteers.
Handling reports and networks			
China	Zhengzhou flood spreadsheet (2021)	A student's shared spreadsheet of people awaiting rescue, sorted by urgency; over 2.5 million views and 20,000 edits (Tencent's figures); later run by a professional team.	People triage by vulnerability on their own; spreadsheets break at scale.
India	Kerala Rescue (2018)	During the Kerala floods, volunteers built a help-request site in about 14 hours with Django; it became the official government site; its code is public.	Closest to what we build, made at hackathon speed. Read its code.
Japan	171 Dial and 00000JAPAN	Dial 171 from any phone to leave an "I'm safe" voice message; carriers open free Wi-Fi ("00000JAPAN") after big disasters.	Plan for people without smartphones; any Wi-Fi becomes an upload point.
Japan	Honda passable-roads map (2011)	The day after the Tōhoku earthquake, car navigation data showed which roads were still being driven; Google published a version two days later.	Real movement data shows what is still reachable.
USA	Project OWL / ClusterDuck Protocol	Won IBM's first Call for Code (2018); a mesh of cheap radio "Ducks" people reach over ordinary Wi-Fi; piloted in Puerto Rico; open-sourced in 2020.	A mesh idea from a coding challenge became real infrastructure.
Warning	Bridgefy	A mesh messaging app used at protests. Researchers showed in 2020, and again in 2022, that users could be tracked and messages faked.	Sign and seal properly; don't over-claim security.

PART 7

Risks and the day itself

7.1 Risks and backups

The two biggest risks are the venue's Wi-Fi and Bluetooth on stage. We handle both by bringing our own router and keeping the QR backup ready. The biggest build risk is scope: we protect the Must list and cut from the bottom.

Risk	Likelihood	Backup
Bluetooth discovery is flaky in a crowded hall	High	Rehearse in a busy room; QR backup; recorded backup video.
Venue Wi-Fi blocks phone-to-laptop traffic	High	Bring our own router, or a phone hotspot the laptop joins.
Laptop firewall blocks phones	High	Allow port 8000 on private networks; test it in hour 0.
Scope creep (47 features, many disasters)	High	Two profiles only until hour 28; cut from the bottom; freeze at hour 33.
A teammate gets stuck	High	The 45-minute rule: ask or switch; pair up.
Team positions arrive late through the mesh	High	Show "as of" times, grey stale icons, overdue and not-heard-from flags; the dispatcher can set a status by hand after a call.
Location pings drain team phone batteries	Medium	Ping every 2 minutes only on a job, 10 minutes when idle; a power bank on every team.
Nearby Connections won't work on our phones	Medium	Try another pair of phones (the same model helps); fall back to QR-only relay.
Read-aloud voice missing for a language	Medium	Install the offline voice in advance; fall back to English or show text only.
The AI service is slow or down	Medium	Keyword reader; AI results for scenario messages saved ahead of time.
The map doesn't load offline	Medium	`.pmtiles` file; else open the area once online to cache it; else a plain background with zone outlines.
A merging bug creates one giant case	Medium	Tests, the fixed 150 m limit, and the Split button.
A phone battery dies on stage	Low	Power bank and cables on the table; screens dimmed.
Judges ask "is this only for floods?"	Certain	The live heatwave switch, plus the earthquake profile file.
Judges ask how we warn people without phones	Certain	Door knocks, loudspeaker scripts, outreach lists; Bangladesh and Chicago examples.
Judges ask how we know a team got there	Certain	Automatic "on scene" within 100 m, the Arrived button, overdue and not-heard-from flags.
Judges ask about fake alerts or reports	Certain	Signed alerts, signed packets, rate limits, confidence, the verify badge.

Risk	Likelihood	Backup
Judges ask about privacy of health data	Certain	Opt-in registry, sealed packets, minimal fields, deleted after the crisis.

7.2 Hackathon-day checklist

Bring

- [ ] 3 Android phones plus 1 spare, each with the app installed, the setup QR scanned and offline voices installed
- [ ] Laptop with the server, dashboard, database, offline map and both scenario packs
- [ ] Our own Wi-Fi router, or a spare phone for a hotspot
- [ ] Power bank, chargers, USB cables, power strip
- [ ] HDMI adapter for the projector
- [ ] Printed setup QR and a printed demo script

Rehearsals

- [ ] 5 full runs, each one timed
- [ ] 1 run with the internet cut on purpose
- [ ] 1 run where Bluetooth "fails" and we switch to the QR backup
- [ ] 1 run of the heatwave switch on its own
- [ ] 1 run answering the 5 hardest judge questions

30 minutes before going on stage

- [ ] Run `reset_demo.py --pack flood` to reset the database
- [ ] Start the server and dashboard; dashboard full screen at 125% zoom
- [ ] Phones: battery over 80%, battery saver off, screen timeout 10 minutes, volume up, Bluetooth and location on
- [ ] Relay phones in airplane mode with Bluetooth on and no Wi-Fi network joined (offline)
- [ ] Online phone (Phone C) and laptop on our router
- [ ] One test alert and one test report end to end, then reset again
- [ ] Backup video open in a tab
- [ ] Agree who talks, who clicks, who handles the phones

REFERENCE

Glossary

Term	What it means, simply
Hazard / disaster type	The kind of disaster: flood, heatwave, earthquake, storm or cyclone, wildfire, landslide.
Hazard profile	A small settings file for one disaster type: its questions, time rules, team needs, window rule, tips and alert wording.
Alert	A warning issued by the command post for certain zones, signed so it can't be faked.
Severity: watch, warning, emergency	Be ready / act today / act now.
Alert reply	"I'm safe", "I need help" or "I'm helping others", tapped on the alert screen.
Warning coverage	The share of registered vulnerable people in a zone who are confirmed to have heard the warning.
Outreach list	Who volunteers should visit or call first, highest risk first.
Announcement script	A 20-second text for loudspeakers, megaphones or radio, in local languages.
Cell broadcast	A way for phone networks to send an alert to every phone in an area without an app (used by FR-Alert and Japan's earthquake warnings).
Text-to-speech	The phone reading text out loud. Works offline with installed voices.
Mesh network	Devices talk directly to each other instead of through a phone tower or the internet.
Store and forward	Keep a copy of a message and pass it on to the next device you meet, like passing a note along a line.
Delay-tolerant network	A network built for messages that may take a while to arrive, because the connection comes and goes.
Nearby Connections	Google's Android system for finding nearby phones and sending data over Bluetooth and Wi-Fi, with no internet.
Online / offline phone	Online: can reach the server over Wi-Fi or mobile data, so it sends directly. Offline: can't, so it relays phone to phone.
Packet	One message travelling through the mesh: a readable outside plus a sealed (or, for alerts, public) inside.
Sealed box	A way of encrypting so that only the holder of one secret key (the command post) can open it.
Digital signature	A mathematical seal that proves who made a message and that nobody changed it.
Public / private key	A matching pair: the public key can be shared to seal things or check signatures; the private key stays secret to open or sign.
Geohash	A short code for an area on the map. Five letters cover about a 5 km box.
Hop	One step of a packet from one phone to the next.
Case	One place or person that needs help. Several reports can merge into one case.

Term	What it means, simply
Hours to critical	Our estimate of how long until a situation becomes life-threatening.
Window closing	The moment help gets much harder to deliver: a road floods, landfall, the hottest hours, night.
Priority points	The score that decides the order of the list. Higher goes first.
Confidence	A number from 0 to 1 for how much we trust a case.
Hard-hit zone	A zone the active profile counts as affected (flood extent, heat-alert area, shaking area...).
Silent Zone	A hard-hit area sending far fewer reports or replies than expected, so we send someone to check.
Registry	An opt-in list of people who may need help, like those on oxygen, in a wheelchair or living alone.
Welfare check	Someone going to see if a person or area is OK.
Cooling centre	A cool public place (hall, library, clinic) open to people during a heatwave.
Heatstroke / heat exhaustion	Serious heat illness. Heatstroke (confusion, fainting, hot dry skin) is an emergency; heat exhaustion (dizziness, vomiting, cramps) can lead to it.
Search and rescue	Teams trained and equipped to find and free people trapped in collapsed buildings or debris.
Team status	Where a team is in its job: available, assigned, en route, on scene, returning, or out of service.
Location ping	A small message a team phone sends every few minutes with its position, battery and fuel.
Last seen	The time of a team's most recent known position.
Overdue	A team that hasn't arrived by its expected time plus a margin.
Outcome form	What a team reports when a job is done: people helped, where they went, supplies used.
Supply inventory	How much of each supply is at the command post, at each shelter and on each team.
Burn rate	How fast a supply is being used, used to predict when it runs out.
Triage	Deciding who gets help first when there isn't enough for everyone at once.
Dispatcher / call-taker	The people at the command post who decide which team goes where, and who type in reports from calls and radio.
First-come-first-served (FCFS)	Helping people in the order their requests arrived.
AI reader / LLM	A large language model that turns messy text into clear fields. In PulseTriage it only reads; it never ranks.
Keyword reader	Simple word lists that do the AI's job, less well, when there is no internet.
API	The set of web addresses the apps use to talk to the server, like `/api/cases`.
Flutter, FastAPI, SQLite, React, Leaflet	The tools we build with: phone app (Flutter), server (FastAPI), database file (SQLite), web dashboard (React) and maps (Leaflet).

Term	What it means, simply
PMTiles	A single file holding map images for an area, so the map works offline.
Walking skeleton	The thinnest version of the whole system working end to end, before adding features.
Gate	A test at the end of a build phase that must pass before moving on.

Sources

Facts about other systems come from these pages, checked on 17 October 2026, except where marked "not re-checked". The problem statement comes from the CODE//CONNECT'26 problem statements document. Hazard rules and numbers in this brief are our own demo defaults, not taken from these sources.

- Zhengzhou flood spreadsheet: The China Project; Tencent
- Chicago heatwaves: Encyclopedia of Chicago; Christian Science Monitor (1995); account of the 1999 response
- Ahmedabad Heat Action Plan: Eos; Exemplars in Global Health
- Bangladesh Cyclone Preparedness Programme: PreventionWeb / American Red Cross (Cyclone Amphan); Devex; Atlas of disaster risk
- Japan earthquake early warning: Natural Hazards and Earth System Sciences (2020)
- Japan evacuation-support lists: Cabinet Office summary (DINF); NIPH journal (2021)
- 171 Disaster Emergency Message Dial: NTT West; 00000JAPAN: NTT-BP
- Honda passable-roads map: Honda
- Smart911: Smart911 FAQ
- Project OWL / ClusterDuck Protocol: IBM Call for Code; Linux Foundation (March 2020)
- SAIP and FR-Alert: Acteurs Publics
- LizaAlert: Wikipedia; The Moscow Times (2015)
- EMERCOM and OKSION: EMERCOM, Information Center OKSION; EMERCOM mobile app
- Kerala Rescue: GitHub repository; PyCon India 2019 talk proposal
- Bridgefy: Royal Holloway (2020); USENIX Security 2022
- France heatwave registers: not re-checked for this brief.